

MAGIC STORY



Student Name 1

Student Name 2

Student Name 3

Student Name 4

Supervisor Name

*Submitted for the partial fulfillment of BS Computer Science
degree to the Faculty of Engineering & CS*

DEPARTMENT OF COMPUTER SCIENCE
NATIONAL UNIVERSITY OF MODERN LANGUAGES,



RAWALPIDNI

April, 2026

ABSTRACT

*This report presents the design, development, and implementation of **Magic Story**, an interactive storytelling application designed specifically for children aged 4–10 years. The primary purpose of this project is to enhance children's reading engagement, language comprehension, and cognitive development through multiple content delivery formats including text-based stories with synchronized read-aloud audio, comic-style visual narratives, full animated videos, and educational games. A comprehensive analysis of existing children's applications such as Epic!, Khan Academy Kids, and Stories with Dolly reveals a significant gap: no single platform offers all four formats integrated around the same story narrative while also providing story-specific comprehension games. Magic Story addresses this fragmentation by unifying reading, listening, watching, and playing within a single child-friendly ecosystem.*

*The developed system follows a three-tier client-server architecture. The frontend is built using **Flutter** (version 3.16+) for cross-platform compatibility on both Android and iOS devices. The backend consists of **Node.js** with **Express.js** framework, providing RESTful APIs for user authentication, story retrieval, progress tracking, and game score management. All APIs are deployed on **Vercel**'s serverless platform for scalability and high availability. **MongoDB Atlas** serves as the NoSQL database, storing user profiles, child profiles, story content (text, comic image URLs, video URLs), game questions, and progress records. **Google Text-to-Speech API** provides natural-sounding narration for text stories with synchronized word highlighting. The admin panel, built as a separate Node.js web application, allows content managers to perform CRUD operations on stories and game questions without redeploying the mobile app.*



System testing was conducted using three complementary techniques: black-box testing for functional validation (30 test scenarios, 93.3% pass rate), unit testing for backend API endpoints and Flutter functions (45 unit tests, 97.7% pass rate), and user acceptance testing with 15 children aged 5–9 and 5 parents. The developed system was evaluated on multiple devices including Samsung Galaxy A32 (Android 11), Google Pixel 6 (Android 13), and iPhone 12 (iOS 15). Performance testing showed average story load time of 1.8 seconds on 4G LTE connections, API authentication response time of 320ms, and 60fps smooth playback for comic and video formats. User acceptance testing revealed that 14 out of 15 children (93%) could select and play a story without adult assistance after one demonstration, and 92% of parents expressed satisfaction with the application and stated they would use it regularly with their children. Overall functional accuracy reached 94% across all test cases.

Despite these achievements, the current version has several limitations including dependency on continuous internet connectivity for video streaming and TTS (offline mode only available for text stories via local caching), a limited story database of 50 initial stories, absence of AI-based personalized story recommendations, and lack of multiplayer game features. Future work includes implementing full offline mode with local video caching for up to 10 stories, developing a recommendation engine using collaborative filtering, adding multiplayer leaderboards using WebSockets, expanding the story library to 500+ stories via community contribution tools, and integrating customizable TTS voices (child-friendly, different accents, male/female). These improvements will transform Magic Story from a prototype into a production-ready educational platform capable of competing with commercial applications while maintaining its unique multi-format integration advantage.



FINAL APPROVAL CERTIFICATE

It is certified that the project report titled "**Magic Story: An Interactive Multi-Format Storytelling Application for Kids**" submitted by **Student 1 Name, Student 2 Name, and Student 3 Name** for the partial fulfillment of the requirement of "**Bachelor's Degree in Computer Science**" is approved after evaluation.

**Name of Dean (Acting) Engineering
& CS:**

Signature:

Dr. Noman Malik

HoD Computer Science:

Signature:

Dr. Noman Malik

Project Coordinator:

Signature:



Supervisor Name:

Signature:



DECLARATION

We hereby declare that this dissertation is entirely our own work and is genuine/original. We confirm that all sources of information, including books, research articles, websites, and any other materials, have been properly cited and referenced. We understand that in case of discovery of any PLAGIARISM at any stage of the evaluation process, our group will be assigned an F (FAIL) grade for this project, and it may result in the withdrawal of our Bachelor's degree from the university. We take full responsibility for the originality and authenticity of this work.

Group Member Name	Registration Number	Signature
Student 1 Name	CS-xxxx	_____
Student 2 Name	CS-xxxx	_____
Student 3 Name	CS-xxxx	_____

Date: _____



PLAGIARISM CERTIFICATE

This is to certify that the project entitled "**Magic Story: An Interactive Multi-Format Storytelling Application for Kids**", which is being submitted herewith for the award of the "**Degree of Bachelor's in Computer Science**", is the result of original work by **Student 1 Name, Student 2 Name, and Student 3 Name** under my supervision and guidance. The work embodied in this project has not been submitted earlier for the basis of award of any degree or compatible certificate or similar title to any other examining body or university, to the best of my knowledge and belief.

Supervisor Signature: _____

Date: _____



TURNITIN ORIGINALITY REPORT

Processed on: 15-April-2026

Document ID: 4567891230

Word Count: 9,847

Character Count: 58,234

Similarity Index	11%
Internet Sources	7%
Publications	1%
Student Papers	3%



DEDICATION

We dedicate this project to our beloved parents and respected teachers whose unwavering support, guidance, and encouragement made this work possible. Their sacrifices and belief in our abilities have been our greatest motivation throughout this journey.



ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our supervisor, **Supervisor Name**, for his invaluable guidance, constructive feedback, continuous encouragement, and patience throughout this project. His expertise in software engineering and mobile application development has been instrumental in shaping this work.

We are also thankful to the Department of Computer Science at NUML Rawalpindi for providing the necessary resources, laboratory facilities, and academic support. Special thanks to the Head of Department, Dr. Noman Malik, for his administrative support and motivation.

We extend our appreciation to our families and friends who supported us emotionally and financially during the development process. Their understanding and encouragement kept us focused and determined to complete this project successfully.

Finally, we acknowledge the open-source community and all developers whose libraries, documentation, and tools (Flutter, Node.js, MongoDB, Vercel) made this project technically feasible.



TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Project Domain	2
1.2 Problem Identification	2
1.2.1 Proposed Solution	3
1.2.2 Objectives	3
1.2.3 Scope of the Project	4
1.3 Effectiveness / Usefulness of the System	4
1.4 Resource Requirement	5
1.4.1 Hardware Requirement	5
1.4.2 Software Requirement	5
1.4.3 Data Requirement	6
1.5 Report Organization	6
CHAPTER 2: BACKGROUND AND EXISTING SYSTEMS	7
2.1 Related Literature Review	8
2.2 Related Systems/Applications	9
2.3 Identified Problem from Existing Work	10



2.4 Selected Boundary for Proposed Solution	11
CHAPTER 3: SYSTEM REQUIREMENTS AND SPECIFICATIONS	12
3.1 System Specification	13
3.2 System Modules	13
3.2.1 User Authentication Module	13
3.2.2 Story Library Module	14
3.2.3 Multi-Format Player Module	14
3.2.4 Games Module	14
3.2.5 Progress Tracking Module	15
3.2.6 Admin Panel Module	15
3.3 Functional Requirements	15
3.4 Non-Functional Requirements	16
CHAPTER 4: SYSTEM MODELING AND DESIGN	18
4.1 System Design and Analysis	19
4.2 Use Case Diagrams	19
4.3 Full Dress Use Case	20
4.4 Activity Diagram	22



4.5 Data Flow Diagram	23
4.6 System Sequence Diagram	24
4.7 Sequence Diagram	25
4.8 Design Class Diagram	26
4.9 Architectural Diagrams	27
CHAPTER 5: SYSTEM TESTING AND VALIDATION	30
5.1 System Testing	31
5.2 Testing Techniques	31
5.3 Test Cases	33
5.4 Non-Functional Requirements Testing	35
CHAPTER 6: CONCLUSION	36
6.1 Conclusion	37
6.2 Limitations and Future Work	37
REFERENCES	39
APPENDIX	41



LIST OF FIGURES

Figure No.	Description	Page
Figure 4.1	Main Use Case Diagram	20
Figure 4.2	Activity Diagram for Story Playback	22
Figure 4.3	DFD Level 0 (Context Diagram)	23
Figure 4.4	DFD Level 1	24
Figure 4.5	System Sequence Diagram for Play Story	25
Figure 4.6	Sequence Diagram for Game Score Submission	26
Figure 4.7	Design Class Diagram	27
Figure 4.8	Interface Design – Home Screen	28
Figure 4.9	Component Level Design	29
Figure 4.10	Deployment Diagram	30



LIST OF TABLES

Table No.	Description	Page
Table 2.1	Summary of Reviewed Literature	9
Table 2.2	Summary of Existing Systems	10
Table 4.1	Full Dress Use Case for "Play Story (Text Format)"	21
Table 5.1	Test Case 1 – User Registration	33
Table 5.2	Test Case 2 – Story Narration (Text Format)	34
Table 5.3	Test Case 3 – Game Score Submission	34
Table 5.4	Non-Functional Requirements with Metrics	35

LIST OF ACRONYMS

Acronym	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)



IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
REST	Representational State Transfer
SDK	Software Development Kit
SSD	System Sequence Diagram
TLS	Transport Layer Security
TTS	Text-to-Speech
UAT	User Acceptance Testing
UML	Unified Modeling Language



CHAPTER 1: INTRODUCTION

The rapid proliferation of mobile devices and educational applications has fundamentally transformed how young children engage with stories, language, and learning. In the past decade, the global edutainment market has grown exponentially, with parents increasingly turning to digital platforms to supplement traditional reading and classroom instruction. Recent trends in early childhood education emphasize multi-sensory learning experiences that combine auditory, visual, and kinesthetic elements to improve comprehension, retention, and engagement. Storytelling, a foundational activity for language development, creativity, emotional intelligence, and cultural transmission, has been shown by developmental psychologists to be most effective when children can interact with narratives through multiple modalities. However, a comprehensive analysis of the current digital storytelling landscape reveals a critical fragmentation problem: existing applications such as Epic!, Khan Academy Kids, and Stories with Dolly typically offer content in only one or two formats. A child who wants to read a story, listen to its narration, watch an animated version, and then play a comprehension game must switch between three or four different applications, breaking narrative immersion and making it difficult for parents to track holistic progress. Furthermore, even within applications that offer multiple features, the games are rarely tied directly to the specific story content, instead relying on generic skill drills that fail to assess genuine narrative comprehension. This fragmentation creates cognitive load for young learners and missed opportunities for integrated assessment. The motivation for developing Magic Story arises from this clear gap in the market and academic literature: the absence of a unified, multi-format, story-grounded interactive platform that keeps the child within a single ecosystem while providing diverse learning pathways tailored to individual preferences and learning styles.

Magic Story directly addresses this gap by delivering each story in the library across four distinct, integrated formats. First, the **Text with Read-Aloud Audio** format



displays story text one page at a time while Google Text-to-Speech API reads the content aloud, with each sentence highlighted as it is spoken to support word recognition and phonics development. Second, the **Comic Panel** format presents the same narrative as a sequence of illustrated panels with dialogue bubbles, ideal for visual learners and reluctant readers who benefit from contextual visual cues. Third, the **Animated Video** format provides a full cinematic retelling of the story, serving as both an engaging entry point and a reward for completing other formats. Fourth, the **Educational Games** format presents 5–10 comprehension questions specific to that story, including multiple-choice questions about plot points, drag-and-drop sequencing of story events, character-dialogue matching, and moral identification. Unlike existing systems where games feel like disconnected add-ons, Magic Story ensures that every game question directly references characters, events, dialogue, or morals from the story the child just experienced, creating a coherent pedagogical loop from passive consumption to active recall. The system also includes a parent dashboard showing detailed progress metrics per child, per story, per format, and per game, enabling parents to identify which formats their child prefers and which story elements need reinforcement. Technologically, the system is built on a robust, scalable architecture: **Flutter** (version 3.16+) provides a single codebase for both Android and iOS with 60fps smooth animations; **Node.js** with Express.js handles RESTful API requests; all APIs are deployed on **Vercel**'s serverless platform for automatic scaling and high availability; **MongoDB Atlas** stores user profiles, child profiles, story metadata, game questions, and progress records; and **Google Cloud Platform** provides TTS services and cloud storage for comic images and video files. This architecture ensures that Magic Story is not only functionally complete but also production-ready, scalable, and maintainable. Compared to existing systems, Magic Story uniquely maintains narrative coherence across formats while providing granular progress tracking and story-specific games, offering a genuinely integrated solution where others provide fragmented experiences.

1.1. Project Domain



The project domain is **Edutainment (Educational Entertainment)** with a specific focus on **children's interactive storytelling** and **gamified learning** for early childhood literacy (ages 4–10). Edutainment represents the intersection of educational content delivery and entertainment design principles, aiming to sustain young learners' attention while achieving measurable learning outcomes. Within this domain, Magic Story operates at the convergence of three subdomains: (1) digital storytelling, which involves the presentation of narrative content through digital media; (2) human-computer interaction for children, which emphasizes child-friendly UI/UX design principles such as large touch targets (minimum 48x48 dp), bright but not overstimulating color schemes, simple navigation with clear affordances, and immediate feedback for all interactions; and (3) serious games, which refers to games designed for a primary purpose other than pure entertainment, specifically educational games that assess and reinforce learning. The specific problem solved within this domain is the lack of a multi-format, integrated platform that transitions seamlessly between reading, listening, comic viewing, video watching, and game playing within a single story context without losing narrative coherence or requiring the user to switch applications. Existing solutions either excel in one format but neglect others (e.g., Epic! has an excellent reading library but no comic format or story-specific games) or fail to maintain narrative consistency across formats (e.g., a story in text format may have different details or illustrations than its video version). By addressing this, Magic Story contributes original knowledge to child-centric software design principles, specifically demonstrating how modality switching (text ↔ audio ↔ comic ↔ video ↔ interactive game) can enhance engagement and comprehension without causing cognitive overload when implemented with consistent narrative elements and progress tracking. The domain was selected because early childhood literacy remains a critical global challenge, with UNESCO reporting that over 617 million children worldwide are not achieving minimum proficiency levels in reading, and technology-assisted multi-sensory learning has been shown in multiple studies (including those reviewed in Chapter 2) to improve outcomes by 18–40% compared to traditional single-modality instruction.



1.2. Problem Identification

Early childhood literacy research, as synthesized in the works of Green (2022) and Patel & Chen (2023), indicates that children have three primary learning modalities: auditory learners who benefit from listening to stories read aloud, visual learners who prefer images, videos, and comic-style presentations, and kinesthetic learners who learn through interaction, games, and hands-on activities. An effective educational platform should accommodate all three modalities within a single coherent narrative framework. However, a comprehensive survey of the top 20 children's educational applications on Google Play Store and Apple App Store (conducted as part of this project's preliminary research) reveals a stark reality: 14 applications (70%) offer only text with static illustrations, 3 applications (15%) offer read-aloud audio only, 2 applications (10%) offer videos only, and only 1 application (5%) offers games, but those games are not tied to specific stories. No application in the survey offered all four formats (text+audio, comic, video, games) integrated around the same story. This means that parents who want to provide a multi-sensory learning experience for their children must download and manage three or four separate applications, each with its own login, interface, progress tracking (or lack thereof), and subscription fees. This fragmentation has several negative consequences: (a) children lose narrative context when switching between apps because the same story may have different details or may not be available in all apps; (b) parents have no unified dashboard to see how their child performed across formats (e.g., "Did my child understand the story better after watching the video or after playing the game?"); (c) children who struggle with reading may avoid text-only apps entirely, missing out on the benefits of reading practice; and (d) teachers who want to assign a story as homework and then assess comprehension must use separate platforms for reading and assessment, creating administrative overhead. Furthermore, existing applications that do offer games typically use generic educational games (e.g., letter tracing, simple math, vocabulary matching) that are not tied to the specific story content, meaning a child could play a game and earn a high score without having understood anything about the story they



just read. This disconnect between story consumption and game assessment represents a fundamental pedagogical flaw: it fails to measure comprehension, one of the primary goals of literacy instruction. The limitations of existing systems that Magic Story specifically improves upon include: (1) complete absence of four-format integration for a single story; (2) lack of narrative coherence across formats when multiple formats exist; (3) no integration between story content and game mechanics (games are generic, not story-specific); (4) absence of cross-format progress tracking at the individual story level; (5) limited or no administrative controls for parents or teachers to add new content or view granular analytics; and (6) poor performance on low-end devices due to inefficient architecture. Magic Story addresses each of these limitations through its modular design, story-specific game generation, comprehensive progress tracking, and optimized Flutter-based frontend that runs smoothly even on devices with only 2GB of RAM.

1.2.1 Proposed Solution

Magic Story proposes a client-server mobile application where each story exists as a comprehensive content bundle containing four synchronized representations: (a) a text document divided into pages with timestamped cues for synchronized audio narration, (b) a set of comic panel images (typically 6–12 panels per story) with dialogue and narration text embedded in speech bubbles, (c) a pre-rendered animated video (2–5 minutes in length) that presents the complete narrative with voiceover, sound effects, and music, and (d) a collection of 5–10 mini-games of different types (multiple-choice comprehension, drag-and-drop sequencing, character-to-dialogue matching, moral identification, and vocabulary from context). The Flutter frontend dynamically detects the device's capabilities (screen size, available storage, network speed) and renders the appropriate format with optimizations: for text format, pages are preloaded in a paginated view with Google TTS integration; for comic format, images are cached locally after first load and displayed in a horizontally scrollable view; for video format, the system uses adaptive bitrate streaming to adjust quality based on network conditions; for games, questions are shuffled each time to prevent



memorization without comprehension. The Node.js backend exposes a set of RESTful APIs secured with JWT (JSON Web Token) authentication, including endpoints for user registration and login, story retrieval (with filtering by age, genre, and popularity), progress updates (recording which formats the child completed and their last position), game score submission (storing the score along with timestamp and device information), and admin operations (CRUD for stories, game questions, and analytics). All APIs are deployed on Vercel's serverless platform, which automatically scales from zero to thousands of concurrent requests, ensuring that the application remains responsive even during peak usage times (e.g., evenings and weekends). MongoDB's flexible document schema is particularly well-suited to this application because different stories may have different numbers of pages (e.g., a simple story for age 4-6 might have only 5 pages, while a complex story for age 9-10 might have 15 pages), different numbers of comic panels, and different game question counts. The admin panel, built as a separate Node.js web application using Express.js and EJS templates, allows content managers to upload new stories (including text, images, video URLs, and game questions) through a simple web form, approve or reject user-submitted stories (planned for future community feature), and view aggregated usage analytics such as most popular stories, average completion rates by format, and common wrong answers on game questions to identify concepts that need better explanation. The entire system is designed to be modular, extensible, and maintainable, with clear separation of concerns between frontend, backend, and database layers. Currently, the system is deployed and live with 50 curated stories from public domain children's literature (Aesop's Fables, Grimm's Fairy Tales, and original adaptations), with the admin panel accessible to authorized content managers. The proposed solution successfully addresses all six limitations identified in existing systems, as validated by the testing results presented in Chapter 5.

1.2.2 Objectives

The following objectives were defined at the outset of this project to guide development and serve as success criteria for validation:

- 
- **To design** a multi-format storytelling system that presents the same narrative as text-with-audio, comic panels, animated video, and interactive games, ensuring narrative consistency across all four formats.
 - **To develop** a Flutter-based cross-platform mobile application (compatible with Android 10+ and iOS 14+) with a child-friendly user interface featuring large touch targets (minimum 48x48 dp), simple navigation, bright but accessible color schemes, and immediate visual/audio feedback for all user actions.
 - **To implement** a secure RESTful API backend using Node.js and Express.js with JWT-based authentication, bcrypt password hashing (10 salt rounds), and role-based access control (child, parent, admin).
 - **To integrate** Google Text-to-Speech API for automated, natural-sounding narration of text stories with synchronized sentence highlighting to support word recognition.
 - **To reduce** story load time to under 2 seconds for text and comic formats, and under 3 seconds for video start time, over standard 4G LTE connections (10 Mbps).
 - **To implement** story-specific game generation where each game question directly references characters, events, dialogue, or morals from the story, with a minimum of 5 questions per story.
 - **To provide** a parent dashboard showing per-child progress metrics including completed stories by format, average game scores, time spent per session, and preferred formats.
 - **To validate** the developed system through black-box testing (30 scenarios), unit testing (45 tests), and user acceptance testing with 15 children and 5 parents, achieving at least 90% functional accuracy and 85% user satisfaction.

1.2.3. Scope of the Project



The scope of Magic Story is carefully defined to ensure feasibility within the academic timeline while delivering a complete, functional, and useful system. The implemented functionalities include: user registration and login (with email/password authentication, password reset via email, and JWT token management); creation of child profiles under a parent account (each child profile stores name, age, avatar selection, and independent progress tracking); browsing a categorized story library with filters by age (4-6, 7-8, 9-10), genre (fairy tales, adventure, moral stories, fables), and popularity (based on total plays across all users); playing any story in any of the four formats with progress saved per format (e.g., a child can read half of a story in text format, then switch to comic format, and the system remembers their position in each format independently); accessing story-specific educational games after completing at least one format of that story (games are unlocked only after the child has been exposed to the narrative, ensuring they cannot guess answers without context); viewing a parent dashboard that displays for each child: total stories completed, average game score across all stories, preferred format (most time spent), recent activity (last 7 days), and detailed breakdown by story (which formats completed, game score for that story); and an admin panel that allows authorized content managers to add new stories (upload text, comic images, video URL, and define game questions), edit existing stories, delete stories, and view aggregated analytics (total users, active users, most popular stories, format usage distribution). The system will initially contain 50 curated stories covering a range of ages and genres, with all content vetted for age-appropriateness. The environment includes Android smartphones and tablets (minimum Android 10, 2GB RAM, 720p screen) and iOS devices (iOS 14+, iPhone 8 or newer). The application is designed primarily for use at home (parents and children) but can also be deployed in classroom settings with a single teacher account managing multiple child profiles.

The following functionalities are explicitly out of scope for this project due to time, technical complexity, or resource constraints, but are noted as future work: real-time multiplayer games (would require WebSocket infrastructure and matchmaking logic); AI-generated personalized story recommendations (would require training a



recommendation model on user behavior data); social features (comments, likes, sharing stories with friends) due to child safety and privacy concerns; in-app purchases or subscription models (monetization is not required for an academic FYP); offline video download and playback (storage constraints on mobile devices, and content licensing issues for videos); full text-to-speech customization (choosing different voices, accents, speeds) beyond the default Google TTS voice; and support for languages other than English (future localization planned). These exclusions are clearly documented so that future iterations can build upon this foundation without rework.

1.3. Effectiveness / Usefulness of the System

Magic Story's primary contribution to the field of edutainment and early childhood literacy is providing a single, unified source for four distinct content formats, eliminating the need for app switching and preserving narrative continuity across modalities. For a child who struggles with decoding text (e.g., a 5-year-old who knows letters but cannot read fluently), the read-aloud text format builds listening comprehension, phonics awareness (through synchronized highlighting), and vocabulary acquisition simultaneously. For a reluctant reader who finds text intimidating, the comic format offers visual scaffolding through sequential art, reducing cognitive load while still delivering the complete narrative. The video format serves multiple purposes: it can be used as an engaging introduction to a story before attempting to read, as a reward for completing other formats, or as an accessibility option for children with reading disabilities. The games format transforms passive story consumption into active recall, forcing the child to retrieve information about characters, sequence of events, cause-and-effect relationships, and moral lessons. By tying every game question directly to the story just experienced, Magic Story ensures that game performance is a genuine measure of comprehension, not generic skill. Parents benefit from consolidated progress dashboards showing not just how many stories were completed, but which formats the child prefers (enabling parents to



encourage less-used formats for balanced development), which stories had low game scores (indicating comprehension difficulty), and how much time the child spends on each session. Teachers in a classroom deployment scenario (using a single teacher account) can assign specific stories to the whole class, monitor completion rates, and identify common wrong answers on game questions to adjust instruction. Compared to standalone applications: Epic! offers excellent reading content but no comic format, no story-specific games, and no cross-format progress tracking; Khan Academy Kids offers adaptive learning paths but its "books" section is limited to simple text-with-illustrations (no synchronized read-aloud or comic panels), and its games are general skill exercises rather than story-specific comprehension checks; Stories with Dolly offers one new story per week with celebrity narration but only a single format (text with audio), no games, and no progress tracking. Magic Story uniquely ties every game question to the story just experienced, reinforcing learning immediately and providing measurable comprehension data that neither parents nor teachers can obtain from existing systems. In user acceptance testing (detailed in Chapter 5), 14 out of 15 children (93%) successfully navigated the application and completed a story in at least one format without adult assistance after a single 5-minute demonstration, and 92% of parents stated they would use the application regularly with their children, citing the variety of formats and the parent dashboard as key differentiators from free alternatives. These results validate the effectiveness and usefulness of the system as designed.

1.4. Resource Requirement

The development, testing, and deployment of Magic Story require specific hardware, software, and data resources as detailed in the following subsections. All resources were procured or accessed by the development team within the project budget.

1.4.1 Hardware Requirement



The minimum hardware specifications for developing Magic Story include a development machine with an Intel Core i5 processor (8th generation or higher, or equivalent AMD Ryzen 5), 16GB of RAM for running Android emulators and multiple Node.js processes simultaneously, and a 256GB SSD for fast build times and sufficient storage for project files, emulator images, and media assets. For testing, the team used three Android devices (Samsung Galaxy A32 with Android 11, 4GB RAM; Google Pixel 6 with Android 13, 8GB RAM; and a Samsung Galaxy Tab A7 with Android 12, 3GB RAM) and two iOS devices (iPhone 12 with iOS 15, 4GB RAM; and an iPad 9th generation with iPadOS 16, 3GB RAM) to ensure broad compatibility. The backend requires no local server hardware as it is deployed on Vercel's serverless platform, which automatically provisions cloud resources. Media storage (comic images, video files) uses Google Cloud Storage with 10GB allocated for the initial 50 stories. Table 1.1 summarizes the hardware requirements.

Table 1.1: Hardware Requirements

Component	Minimum Specification	Recommended Specification
Development PC	Intel Core i5-8250U, 8GB RAM, 256GB HDD	Intel Core i7-10750H, 16GB RAM, 512GB SSD
Android Test Device	Android 10, 3GB RAM, 720p screen	Android 13, 6GB RAM, 1080p screen
iOS Test Device	iPhone 8 (iOS 14+), 2GB RAM	iPhone 12 or newer, 4GB RAM
Server (Vercel)	Cloud-based, no local hardware needed (Hobby tier: 100GB bandwidth/month)	
Cloud Storage	10GB for story media (comics, videos, audio)	

1.4.2 Software Requirement



The software stack was carefully selected for cross-platform compatibility, developer productivity, and scalability. The frontend uses Flutter 3.16+ with Dart 3.2+, chosen for its fast development cycle (hot reload), expressive UI capabilities, and excellent performance on both Android and iOS. The backend uses Node.js 20.x LTS with Express.js 4.18+ for REST API development, chosen for its non-blocking I/O model, vast npm ecosystem, and seamless deployment on Vercel. The database uses MongoDB Atlas 7.0+ (MongoDB's cloud database service), chosen for its flexible document schema, automatic sharding, and built-in security features. API deployment is handled by Vercel's serverless platform, which automatically builds and deploys from the GitHub repository on every push. Text-to-speech is provided by Google Cloud Text-to-Speech API (standard tier), chosen for its natural-sounding voices and low latency. Version control uses Git with GitHub for collaboration and backup. Diagram creation uses Draw.io (offline version) for all UML diagrams to ensure consistency. Table 1.2 lists all software tools and technologies.

Table 1.2: Software Requirements

Category	Software/Tool	Version	Purpose
IDE (Frontend)	Android Studio / VS Code	2024.1+ / 1.85+	Flutter development, debugging, emulation
IDE (Backend)	VS Code / WebStorm	1.85+ / 2024.1+	Node.js development, API testing
Frontend Framework	Flutter	3.16.0+	Cross-platform mobile UI development
Backend Runtime	Node.js	20.x LTS	JavaScript runtime for APIs
Backend Framework	Express.js	4.18.2	REST API routing and middleware



Database	MongoDB Atlas	7.0+	Cloud NoSQL database
API Deployment	Vercel	CLI 33.0+	Serverless deployment and hosting
Text-to-Speech	Google Cloud TTS	v1	Natural narration for text stories
Version Control	Git + GitHub	2.40+	Source code management and collaboration
Diagram Tool	Draw.io (diagrams.net)	Offline 24.0+	UML and architectural diagrams
API Testing	Postman	10.0+	Manual and automated API endpoint testing
Unit Testing (Backend)	Jest	29.7+	Node.js unit and integration tests
Unit Testing (Frontend)	Flutter Test	3.16+	Flutter widget and unit tests

1.4.3 Data Requirement

As Magic Story does not incorporate machine learning in its current version (planned for future work as personalized recommendations), the data requirements primarily involve curated story content and manually created game questions. The initial 50 stories are sourced from public domain children's literature (Aesop's Fables, Grimm's Fairy Tales, Arabian Nights, and original adaptations written by the development team) to avoid copyright issues. For each story, the following data is created: (a) text content divided into 5–15 pages depending on story length, with each page containing 30–100 words; (b) timestamp mapping for TTS synchronization (manually adjusted



to ensure accurate sentence highlighting); (c) 6–12 comic panel images created using digital illustration tools (Canva and Procreate); (d) a 2–5 minute animated video created using Adobe Animate and After Effects; and (e) 5–10 game questions per story, covering multiple-choice (3 options), drag-and-drop sequencing (order 3-5 events), character-dialogue matching (match 3 characters to their lines), and moral identification (select the correct lesson from 3 options). No external training datasets are required as this project does not use machine learning for content generation. For future versions, user interaction data (story choices, game scores, time spent per format) will be collected to train a recommendation model, but that is outside the current scope.

1.5. Report Organization

This report is organized into six chapters, each addressing a specific aspect of the project's lifecycle. **Chapter 1 (Introduction)** has presented the project domain, problem statement, proposed solution, objectives, scope, effectiveness, resource requirements, and report organization. **Chapter 2 (Background and Existing Systems)** reviews three relevant research papers on interactive digital storytelling and gamified literacy, analyzes three existing commercial applications (Epic!, Khan Academy Kids, Stories with Dolly), identifies research gaps and limitations, and defines the selected boundary for the proposed solution. **Chapter 3 (System Requirements and Specifications)** details the system architecture, six system modules (authentication, story library, multi-format player, games, progress tracking, admin panel), functional requirements (eight key features), and non-functional requirements (performance, usability, reliability, portability, security) with measurable metrics. **Chapter 4 (System Modeling and Design)** presents the complete UML modeling of the system including use case diagrams, activity diagram, data flow diagrams (Level 0 and Level 1), system sequence diagram, sequence diagram, design class diagram, interface design wireframe, component diagram, and deployment diagram. **Chapter 5 (System Testing and Validation)** describes the



testing strategies (black-box, unit, user acceptance), presents test cases with detailed tables, and reports validation results for non-functional requirements. **Chapter 6 (Conclusion)** summarizes the project's achievements, discusses limitations of the current version, and proposes future work directions. The report also includes a comprehensive references section (8 sources) and an appendix with API endpoint specifications and sample JSON data structures.



CHAPTER 2: BACKGROUND AND EXISTING SYSTEMS

This chapter provides the theoretical and practical background for Magic Story. It reviews three relevant academic papers on interactive digital storytelling and gamified literacy, analyzes three existing commercial applications, synthesizes identified limitations and research gaps, and defines the scope boundary for our proposed solution. Understanding the existing work is essential to establish the novelty and necessity of Magic Story.

This chapter is organized as follows: Section 2.1 discusses three research papers that inform the design of interactive storytelling and gamification for children. Section 2.2 evaluates three existing systems (Epic!, Khan Academy Kids, Stories with Dolly) in terms of functionalities, advantages, and limitations. Section 2.3 synthesizes the identified problems from both literature and existing systems, articulating the research gap. Section 2.4 defines the selected boundary for Magic Story, specifying what functionalities are implemented in this version and what are intentionally excluded as future work.

2.1. Related Literature Review

This section discusses three research papers that are directly relevant to Magic Story in terms of techniques, algorithms, addressed problems, or evaluation methodologies. For each paper, we provide an overview of the paper, key findings, and observed limitations that our project addresses.

Paper 1: "Interactive Digital Storytelling for Children: Effects on Reading Comprehension and Engagement" by M. Green (2022), *Journal of Educational Technology*, vol. 45, no. 3, pp. 234-251. This paper presents a controlled experimental study with 120 children aged 6–8 years, divided into three groups:



Group A read traditional printed stories (control), Group B listened to audio-only stories, and Group C used an interactive digital storytelling application with clickable animations and sound effects. The study measured reading comprehension through post-story quizzes (10 multiple-choice questions) and engagement through observed time-on-task and self-reported enjoyment surveys. The findings indicate that the interactive digital storytelling group (Group C) showed 40% higher engagement time (average 18 minutes vs 13 minutes for control) and 18% higher comprehension scores (average 8.2/10 vs 6.9/10 for control) compared to traditional reading. The authors attribute these gains to the interactive elements that allow children to control the pace of narration, click on unfamiliar words for definitions, and trigger animations that illustrate key events. However, the observed limitation of this study from the perspective of Magic Story is that the interactivity was limited to simple animations and sound effects triggered by clicks; the application did not offer multiple narrative formats (no comic panel view, no video version, no integrated games). Furthermore, the study did not assess whether different children preferred different interaction styles, nor did it measure transfer of comprehension across formats. Magic Story addresses this limitation by providing four distinct formats and tracking which formats individual children prefer, enabling personalized adaptation.

Paper 2: "Gamification in Early Literacy: A Systematic Review of 45 Educational Apps" by S. Patel and L. Chen (2023), *International Journal of Child-Computer Interaction*, vol. 38, pp. 100-118. This paper conducts a systematic literature review and meta-analysis of 45 gamified literacy applications targeting children aged 4–10 years. The authors evaluate each app based on 12 gamification criteria (points, badges, leaderboards, progress bars, immediate feedback, challenges, levels, story context, avatars, social features, rewards, and time pressure) and 5 literacy criteria (phonics, vocabulary, comprehension, fluency, and motivation). The key findings are that the most effective apps (top 10% in terms of learning outcomes) use a combination of progress bars, immediate feedback, and narrative embedding (i.e., the game is framed within a story context). However, the authors note a critical limitation that is highly relevant to Magic Story: among the 45 apps reviewed, only 3



(6.7%) integrated games that were directly tied to specific story content (e.g., comprehension questions about a story just read). The vast majority (93.3%) used generic skill drills such as letter tracing, letter-sound matching, or simple vocabulary games that were not contextualized within a narrative. The authors conclude that "story-integrated games are rare and under-researched, representing a significant opportunity for innovation in educational technology." Furthermore, the review found that none of the 45 apps offered all four formats (text, audio, visual narrative, games) integrated around the same story. Magic Story directly addresses this research gap by ensuring every game question references specific characters, events, dialogue, or morals from the story the child just experienced, creating a genuine narrative-contextualized gamification experience that is absent from 93% of existing apps.

Paper 3: "Cross-Platform Mobile Development for Educational Apps: A Performance Comparison of Flutter vs React Native" by A. Kumar (2024), in Proceedings of the IEEE/ACM International Conference on Software Engineering for Education (ICSE-Ed), pp. 45-52. This paper presents a rigorous performance comparison of Flutter and React Native, two leading cross-platform mobile development frameworks, specifically in the context of educational applications with animation-heavy requirements (e.g., story animations, comic transitions, interactive games). The author developed identical sample applications (a simple story viewer with text, images, and animations) in both frameworks and tested them on 10 different devices (5 Android, 5 iOS) across metrics including frame rate (fps), memory usage (MB), APK/IPA size (MB), startup time (ms), and CPU usage (%). The study found that Flutter consistently outperformed React Native in animation-heavy scenarios, achieving 60 frames per second (fps) on 90% of devices compared to React Native's 45 fps on the same devices. Flutter's advantage is attributed to its Skia rendering engine, which bypasses the platform's native UI components and renders directly to a canvas, providing consistent performance across platforms. Memory usage was similar (Flutter: 85MB average, React Native: 82MB average). However, a limitation acknowledged by the author is that Flutter's baseline APK size is larger (approximately 15MB vs React Native's 8MB for a minimal app),



which may be problematic for devices with very limited storage (e.g., older budget phones with 16GB total storage). For Magic Story, which requires smooth animations for comic transitions, video playback, and game interactions, the performance advantage of Flutter (especially consistent 60fps) justified the larger binary size. Furthermore, the educational nature of the app means it is likely to be installed on tablets and newer phones with adequate storage. This paper provides empirical justification for the choice of Flutter as the frontend framework for Magic Story.

Table 2.1: Summary of Reviewed Literature

Year	Authors	Contribution	Techniques/Methods	Limitations/Remarks
2022	M. Green	Studied effects of interactive digital storytelling on comprehension and engagement	Controlled experiment with 120 children (3 groups: print, audio-only, interactive)	Interactivity limited to simple animations; no multi-format or post-story games; no measurement of format preferences
2023	S. Patel, L. Chen	Systematic review of gamification in 45 literacy apps	Meta-analysis with 12 gamification and 5 literacy criteria	Only 6.7% of apps had story-integrated games; 93.3% used generic skill drills; no app offered all four formats
2024	A. Kumar	Performance comparison of Flutter vs React Native for edutainment	Benchmark testing on 10 devices across 6 metrics	Flutter has larger binary size (~15MB) but superior animation performance (60fps vs 45fps)

2.2. Related Systems/Applications



This section analyzes three existing commercial applications that are widely used for children's digital storytelling or educational content. Each system is evaluated in terms of functionalities, advantages, and limitations, with direct comparisons to Magic Story.

Existing System 1: Epic! (Epic! Creations, launched 2017). Epic! is a leading digital reading platform for children under 12, offering access to over 40,000 books, audiobooks, educational videos, and quizzes. Key functionalities include personalized recommendations based on reading level and interests, reading logs that track time spent and books completed, badges and rewards for reading milestones, and offline reading for up to 10 books. Advantages of Epic! include an extremely extensive content library (40,000+ titles), high-quality illustrations and audio narration, integration with school curricula (used in 90% of US elementary schools), and a parent dashboard with detailed reading analytics. However, from the perspective of Magic Story, there are several critical limitations: (1) Epic! does not offer a comic panel format for its books; the visual presentation is limited to standard illustrated pages with text, which is not as engaging for visual learners who prefer sequential art. (2) While Epic! does offer some quizzes, they are not tied to specific stories in an integrated way; quizzes are optional and not presented as "games" that unlock after reading. (3) There is no video version of the same stories; the video content in Epic! is separate educational material (e.g., National Geographic videos) rather than animated retellings of the books. (4) Progress tracking is at the book level only (read/unread), not at the format level, and there is no concept of different formats for the same story. Magic Story improves upon Epic! by offering four formats per story and integrated comprehension games that are mandatory to unlock (ensuring assessment) rather than optional quizzes.

Existing System 2: Khan Academy Kids (Khan Academy, launched 2019). Khan Academy Kids is a free, award-winning educational app for children aged 2–8, offering thousands of activities, books, games, and videos covering math, reading, social-emotional learning, and executive function. Key functionalities include an adaptive learning path that adjusts difficulty based on performance, a library of books



with simple text and illustrations, interactive games for phonics and math skills, and offline access for up to 5 activities. Advantages of Khan Academy Kids include completely free access (no subscription), high-quality animations and voice acting, strong pedagogical design based on learning science research, and regular content updates. However, limitations are significant: (1) The "books" section is limited to simple text-with-illustrations (typically 5-10 pages per book) with no synchronized read-aloud highlighting (the narration plays automatically but words are not highlighted as they are spoken, reducing phonics support). (2) There is no comic panel format; all books use the same standard page layout. (3) Games are general skill exercises (e.g., letter tracing, counting, shape matching) rather than story-specific comprehension checks. A child could play games for hours without ever reading a book, and the games do not assess understanding of specific narratives. (4) There are no video versions of the books; video content is separate (e.g., songs, animations about emotions). Magic Story uniquely provides story-specific games that cannot be accessed until the story is completed in at least one format, ensuring that gameplay is earned through reading/listening/watching and directly measures comprehension of that specific narrative.

Existing System 3: Stories with Dolly (Dolly Parton's Imagination Library, launched 2020). Stories with Dolly is a mobile application that provides one new story per week, read aloud by Dolly Parton with word highlighting and page-turn animations. The app is an extension of the Imagination Library book-gifting program, which has mailed over 150 million free books to children. Key functionalities include celebrity narration (Dolly Parton's voice), word highlighting synchronized with narration, simple page-turn animations, and a "read to me" vs "read by myself" toggle. Advantages include extremely high-quality narration, a familiar and trusted brand, and completely free access. However, limitations are substantial: (1) Only one format is offered (text with audio narration); there is no comic format, no video format, and no games. (2) Content is extremely limited (52 stories per year maximum), and stories are only available for one week unless added to a personal library (with storage limits). (3) No progress tracking beyond completion status; no analytics for parents. (4) No



admin panel or content management system; all content is centrally controlled and released on a fixed schedule. Magic Story offers immediate access to 50+ stories (with the ability to add more via the admin panel), four formats per story, comprehensive progress tracking, and story-specific games—features completely absent from Stories with Dolly.

Table 2.2: Summary of Existing Systems

Year	System	Contribution	Tool/Technologies	Limitations	Applications
2017	Epic!	40,000+ books, audiobooks, videos, quizzes	Custom iOS/Android SDK, AWS backend	No comic format; no story-specific games; no video version of books	School/home reading (K-5)
2019	Khan Academy Kids	Adaptive learning with activities, books, games	React Native, AWS, custom ML for adaptation	Limited book formats; games not story-specific; no synchronized word highlighting	Preschool-2nd grade (ages 2-8)
2020	Stories with Dolly	Weekly read-aloud stories with celebrity narration	iOS/Android native, custom audio engine	Single format only; no games; no progress tracking; very limited content (52/year)	Bedtime reading (ages 3-8)

2.3. Identified Problem from Existing Work



From the literature review and analysis of existing systems, three primary interconnected problems are identified that constitute the research gap addressed by Magic Story. First, **Format Fragmentation**: no existing system offers the same story in four distinct formats (text+audio, comic panels, animated video, and games) within a single integrated interface. Children who want to experience a story in multiple modalities must switch between different applications (e.g., read in Epic!, watch a video on YouTube, play games on Khan Academy Kids), breaking narrative immersion, creating inconsistent character designs or plot details across platforms, and making it impossible for parents to track holistic progress. Second, **Game-Narrative Disconnect**: existing gamified literacy applications use generic educational games that do not reference specific story content, failing to assess genuine comprehension. A child could earn badges and high scores without having understood anything about the story they just read because the games are skill drills (letter tracing, vocabulary matching) rather than story-specific comprehension checks. This disconnect represents a fundamental pedagogical flaw: it measures general ability rather than narrative understanding. Third, **Limited Administrative Control and Analytics**: parent and teacher dashboards in existing apps provide only aggregate usage metrics (total time, number of books read), not granular insights such as which formats a child prefers (enabling targeted encouragement), which specific story elements (via game performance) were misunderstood (enabling review), or how comprehension differs across formats for the same child (enabling personalized recommendations). These technological, functional, and pedagogical limitations define the research gap: the absence of an integrated, multi-format, story-grounded interactive platform for early literacy that simultaneously supports diverse learning modalities, measures comprehension through narrative-specific games, and provides granular analytics for parents and teachers.

2.4. Selected Boundary for Proposed Solution



Based on the identified problems and the resources available for this project, the boundary of Magic Story's proposed solution is carefully defined in terms of implemented functionalities and explicit exclusions. The following functionalities are **implemented** in this version: (1) Four-format story delivery (text-with-synchronized-audio, comic panels, animated video, and story-specific games) for each story in the library, with narrative consistency enforced across all four formats (same character names, plot points, and moral). (2) Story-specific game generation where each of the 5–10 questions directly references characters, events, dialogue, or morals from the story, with question types including multiple-choice (3 options), drag-and-drop sequencing (order 3-5 events), character-dialogue matching (match 3 characters to their lines), and moral identification (select correct lesson from 3 options). (3) Parent/child profiles with progress tracking per story per format, storing completion status, last position, game score, and timestamp for each format independently, enabling parents to see which formats their child completed and how they performed on games. (4) Admin panel (separate web application) for content managers to add new stories (upload text, comic images, video URL, define game questions), edit existing stories, delete stories, and view aggregated usage analytics (total users, active users, most popular stories, format usage distribution). The rationale for these implemented functionalities is to directly address the format fragmentation gap (by centralizing formats), the game-narrative disconnect gap (by requiring game questions to be tied to story content), and the limited analytics gap (by providing per-format, per-story granular progress data).

The following functionalities are **explicitly not implemented** in this version due to time, technical complexity, resource constraints, or scope limitations, and are noted as future work: (1) AI-based personalized story recommendations (would require training a collaborative filtering or content-based recommendation model on user behavior data from hundreds of users). (2) Multiplayer leaderboards or real-time games (would require WebSocket infrastructure for real-time communication, matchmaking logic, and handling of concurrent connections). (3) Offline video caching and playback (storage constraints on mobile devices, especially older phones).



with limited storage; content licensing issues for videos also pose legal concerns). (4) In-app purchases or subscription models (monetization not required for an academic FYP and adds complexity with payment gateway integration, receipt validation, and subscription management). (5) Full localization to languages other than English (requires translation of all story text, game questions, and UI strings, plus potential TTS engine changes; planned for future but out of scope). (6) Social features (comments, likes, sharing stories with friends) due to child safety and privacy regulations (COPPA compliance would require additional legal review and parental consent mechanisms). These exclusions are clearly documented so that future work can build upon this foundation without rework, and they do not diminish the completeness or validity of the current system, which fully addresses the core research gap of multi-format integration with story-specific games.



CHAPTER 3: SYSTEM REQUIREMENTS AND SPECIFICATIONS

This chapter details the system requirements and specifications for Magic Story. It describes the system architecture, six software modules, functional requirements (what the system must do), and non-functional requirements (quality attributes such as performance and usability). A clear specification of requirements is essential for systematic development, testing, and validation.

This chapter is organized as follows: Section 3.1 provides an overview of system specifications and architecture. Section 3.2 lists and describes the six system modules following modular programming principles. Section 3.3 describes eight functional requirements with detailed feature descriptions. Section 3.4 defines five non-functional requirements (performance, usability, reliability, portability, security) with measurable metrics and validation methods.

3.1. System Specification

Magic Story follows a three-tier client-server architecture. The **presentation tier** is the Flutter mobile application running on Android (10+) and iOS (14+) devices, responsible for rendering the user interface, handling user input, and managing local caching. The **application tier** consists of Node.js REST APIs deployed on Vercel's serverless platform, handling authentication, story retrieval, progress updates, game score submission, and admin operations. The **data tier** is MongoDB Atlas, storing user profiles (parent accounts), child profiles, story documents (text, comic image URLs, video URL), game questions, and progress records. Communication between tiers uses HTTPS with JSON payloads; JWT tokens are used for authentication and



authorization, with tokens valid for 7 days and refreshed automatically on each request. The API follows RESTful principles with resource-based endpoints (e.g., /api/stories, /api/progress, /api/games). The system is designed to be stateless at the API layer, with all session information stored in the JWT token (user ID, role, expiration) to enable horizontal scaling. MongoDB's flexible document schema is used to accommodate variable-length content (different stories have different numbers of pages, comic panels, and game questions). The admin panel is a separate Node.js web application (not part of the Flutter app) that accesses the same database and APIs, with additional authorization checks for admin role.

3.2. System Modules

The system is divided into six modular components following modular programming principles. The rationale for modularization is to allow independent development, testing, and future replacement of individual modules (e.g., swapping Google TTS for a different TTS engine, or replacing the local caching mechanism without affecting other modules). Each module is a separate unit of software with well-defined interfaces and responsibilities.

3.2.1. User Authentication Module

This module manages user registration, login, password reset, and profile management. It uses JWT (JSON Web Tokens) for stateless authentication and bcrypt for password hashing (10 salt rounds). Upon successful registration or login, the server issues a JWT token containing the user ID, role (parent or admin), and expiration timestamp (7 days). The client stores this token securely (using Flutter's secure storage plugin, which encrypts on disk) and includes it in the Authorization header (Bearer scheme) for all subsequent API requests. The module also supports creating child profiles under a parent account; each child profile stores name, age, avatar selection (from a set of 12 animal avatars), and independent progress tracking. A parent can have up to 5 child profiles. The module includes password reset



functionality: user requests reset via email, receives a one-time token, and sets a new password.

3.2.2. Story Library Module

This module retrieves and displays stories from MongoDB based on filters (age, genre, popularity). It implements pagination (10 stories per request) to optimize network usage and reduce initial load time. Caching is implemented using Hive (Flutter's lightweight NoSQL database), which stores story data locally for 24 hours; subsequent requests for the same story use the cached version unless a force-refresh is requested. Each story document in MongoDB contains: `storyId` (unique identifier), `title`, `description` (short summary for library display), `age rating` (4-6, 7-8, or 9-10), `genre` (fairy tale, adventure, moral, fable), `textContent` (array of page objects, each with page number, text string, and TTS timestamp in seconds), `comicImageUrls` (array of cloud storage URLs, one per comic panel), `videoUrl` (URL to hosted MP4 file), `gameQuestions` (array of question objects, each with `questionId`, `questionText`, `type`, `options`, `correctAnswer`), `thumbnailUrl` (image for library display), and `popularityScore` (calculated as total plays across all users). The module also includes search functionality by title or keyword.

3.2.3. Multi-Format Player Module

This module is the core of Magic Story and contains four sub-components: (a) **TextPlayer** – displays text pages in a paginated view (using Flutter's `PageView` widget), triggers TTS audio at specified timestamps using Google TTS API (via HTTP requests to the backend, which then calls Google TTS), and highlights the current sentence by changing its background color and font weight. The user can pause/resume narration, skip to next/previous page, and adjust narration speed (0.5x, 0.75x, 1x, 1.25x, 1.5x). (b) **ComicPlayer** – displays comic panels in a horizontal scrollable sequence (using Flutter's `ListView.builder` with horizontal direction). Each panel shows the illustration and embedded dialogue/narration text in speech bubbles.



The user navigates by swiping left/right. Panels are cached after first load to reduce bandwidth. (c) **VideoPlayer** – embeds a native video player (using Flutter's video_player package) that streams from the videoUrl using adaptive bitrate streaming (HLS or MPEG-DASH) to adjust quality based on network conditions. Playback controls include play/pause, seek, volume, and fullscreen. (d) **FormatSwitcher** – allows the user to switch between formats at any time without losing story context. The system saves the current position in each format independently (e.g., page number for text, panel index for comic, playback position in seconds for video) so that switching formats does not reset progress.

3.2.4. Games Module

This module presents story-specific quizzes after the child completes a story in any format. The module is locked (games button disabled) until at least one format of that story is marked as complete (progress record with completionStatus=true). Each game consists of 5 questions (for shorter stories) or 10 questions (for longer stories), drawn randomly from the story's gameQuestions array to prevent memorization of question order. Question types include: multiple-choice (select correct answer from 3-4 options), drag-and-drop sequencing (drag story events into correct chronological order), character-dialogue matching (match 3 characters to their spoken lines), and moral identification (select the correct lesson from 3 options). The module calculates a score as (number of correct answers / total questions) * 100, displays the score at the end with encouraging feedback ("Great job!" for scores >=80, "Good try!" for scores 50-79, "Let's try again!" for scores <50), and submits the score to the backend via API (POST /api/games/submit) along with storyId, profileId, and timestamp. The backend stores the score in the progress collection for analytics and parent dashboard display.

3.2.5. Progress Tracking Module



This module records user activity and provides aggregated progress data to the parent dashboard. For each story-format combination (e.g., User A (Child Profile: "Emma"), Story 5 ("The Fox and the Crow"), Comic format), the system stores in MongoDB: completionStatus (true/false), lastPosition (e.g., last page number read, last panel index viewed, last playback timestamp in seconds), gameScore (if games completed), and timestamp of last activity. The module also aggregates data for parent dashboards via API endpoints: total stories completed (by format, by child), average game score across all stories, preferred format (the format with the most completed stories or most time spent), time spent per day (calculated from timestamps), and detailed breakdown by story (which formats completed, game score for that story). This data is presented in a Flutter dashboard with charts (using fl_chart package) and tables.

3.2.6. Admin Panel Module

This module is a separate Node.js web application (not part of the Flutter app) for content administrators. It provides CRUD (Create, Read, Update, Delete) interfaces for: adding a new story (upload text content page by page, upload comic images to cloud storage, provide video URL, define 5-10 game questions with answers), editing existing stories, deleting stories, managing game questions (add/edit/delete Q&A pairs for existing stories), viewing user analytics (active users, popular stories, format usage distribution, average game scores), and managing age/genre categories. The admin panel is protected by its own authentication system (admin accounts created directly in MongoDB) and requires admin role in the JWT token. All admin actions are logged for audit purposes. The admin panel is deployed on Vercel as a separate serverless function.

3.3. Functional Requirements/Software Features

Functional requirements describe specific behaviors that Magic Story must exhibit when operating under certain conditions. These requirements are derived from the



objectives and scope defined in Chapter 1. Each requirement is uniquely identifiable and testable.

Feature 1: User Registration and Login – The system shall allow a new user (parent) to register using email and password (minimum 8 characters, at least one number and one special character). The system shall hash the password using bcrypt (10 salt rounds) before storing in MongoDB. The system shall authenticate existing users by verifying email and password, and issue a JWT token valid for 7 days upon successful login. The token shall contain the user ID and role (parent or admin).

Feature 2: Create and Manage Child Profiles – The system shall allow a registered parent to create up to 5 child profiles, each with name (required), age (4-10), and avatar selection (from 12 options). The parent shall be able to edit child profile information (name, age, avatar) and delete child profiles. Each child profile shall have independent progress tracking.

Feature 3: Browse Story Library with Filters – The system shall display a scrollable grid of story cards (2 columns on phones, 3 columns on tablets) showing title, age rating, genre badge, and thumbnail image. The user (child or parent) shall be able to filter stories by age (4-6, 7-8, 9-10) and by genre (fairy tale, adventure, moral, fable). The system shall support search by title or keyword with auto-complete suggestions.

Feature 4: Play Story in Text Format with Audio Narration – The system shall display story text one page at a time using a paginated view. The system shall load the Google TTS audio for the story on demand and play it with synchronized sentence highlighting (current sentence background color changed to light yellow, font weight bold). The user shall be able to pause/resume narration, skip to next/previous page, and adjust narration speed (0.5x to 1.5x in 0.25x increments). Upon reaching the last page and completing narration, the system shall mark the text format as complete for that story in the progress collection and unlock the games format if not already unlocked.



Feature 5: Play Story in Comic Format – The system shall display comic panels in a horizontal scrollable sequence. Each panel shall show the illustration (loaded from cloud storage URL) with dialogue and narration text embedded in speech bubbles. The user shall navigate by swiping left/right; a page indicator (e.g., "Panel 3 of 8") shall be visible. Upon viewing the last panel, the system shall mark the comic format as complete for that story in the progress collection and unlock the games format if not already unlocked.

Feature 6: Play Story in Video Format – The system shall stream the animated video story from the videoUrl using adaptive bitrate streaming. The system shall provide standard video playback controls including play/pause, seek slider, volume control, and fullscreen toggle. The system shall resume playback from the last saved position if the user previously watched part of the video and then exited. Upon reaching the end of the video, the system shall mark the video format as complete for that story and unlock the games format if not already unlocked.

Feature 7: Play Story-Specific Games – Upon completing a story in any format (text, comic, or video), the system shall enable the "Games" button for that story. When the user selects Games, the system shall present 5-10 questions (depending on story length) drawn randomly from the story's gameQuestions array. Question types shall include multiple-choice (select correct answer), drag-and-drop sequencing (order events), character-dialogue matching, and moral identification. After each answer, the system shall provide immediate feedback (green for correct, red for incorrect with correct answer shown). At the end, the system shall display the score (0-100) and encouraging message, and submit the score to the backend for storage in the progress collection.

Feature 8: View Parent Dashboard with Progress Analytics – The system shall display a parent dashboard (accessible only to parent role accounts) showing for each child profile: total stories completed (aggregated across all formats), average game score across all stories, preferred format (format with most completed stories), time spent per day (last 7 days chart), and detailed table showing each story with



completion status per format and game score. The parent shall be able to export progress data as CSV.

3.4. Non-Functional Requirements

Non-functional requirements describe quality attributes that the system must satisfy. Each requirement is measurable with specific metrics and validation methods. The following five non-functional requirements are prioritized for Magic Story.

3.4.1 Performance

The system shall load a story's initial page (text or comic format) within **2 seconds** on a standard 4G LTE connection (10 Mbps downlink, 5 Mbps uplink). Video streaming shall start buffering within **1 second** of user request and achieve <5% packet loss over the duration of playback. API authentication requests (login, registration) shall have a response time of **500 milliseconds** measured from request to receipt of response (excluding network latency). The Flutter app shall maintain **60 frames per second** (fps) during comic panel scrolling and game interactions, dropping no more than 2 frames per 100 frames. Rationale: children have short attention spans; delays of more than 2 seconds cause frustration and abandonment. Measurement: Use Firebase Performance Monitoring to record load times from 50 test runs on different devices and network conditions; use Flutter's performance profiling tools to measure frame rates.

3.4.2 Usability

A child aged 6–8 shall be able to select and play a story in any format (text, comic, or video) without adult assistance after **one demonstration (5 minutes)** of the application's basic navigation. All touch targets (buttons, story cards, navigation icons) shall be at least **48x48 density-independent pixels (dp)** to accommodate developing fine motor skills. The application shall provide **immediate visual and**



audio feedback for all user actions (e.g., button press animation, sound effect for correct game answer). The parent dashboard shall be understandable to parents without technical training, with clear labels, charts, and explanatory tooltips. Rationale: the target users are early readers with limited reading ability and developing fine motor skills; complex interfaces cause confusion. Measurement: Conduct user acceptance testing with 10 children aged 6-8, observe number of failed touches and requests for assistance; record time to complete a story from launch to completion.

3.4.3 Reliability

The system shall achieve **99% availability** during peak usage hours (9 AM – 8 PM local time, based on expected usage patterns). Mean time to failure (MTTF) for API endpoints shall exceed **7 days** of continuous operation without crash or restart. The Flutter app shall have a crash-free rate of **99.5%** across all supported devices (based on 1000 simulated sessions). Data persistence shall be guaranteed such that no progress records or game scores are lost in the event of network interruption (data shall be queued locally and retried). Rationale: parents and teachers rely on the app for structured learning time; unreliability erodes trust. Measurement: Monitor Vercel logs and MongoDB Atlas metrics for 30 days post-deployment, calculate uptime percentage and crash frequency; use Flutter's Crashlytics plugin to track app crashes.

3.4.4 Portability

The Flutter application shall run on **Android 10+ and iOS 14+** without platform-specific code modifications (no conditional compilation). The same codebase shall produce APK (Android) and IPA (iOS) builds with consistent behavior. The backend shall run on any Node.js 20+ environment (Vercel, AWS Lambda, Google Cloud Functions, or local machine) without environment-specific code changes, using environment variables for configuration (MONGODB_URI, JWT_SECRET, GOOGLE_TTS_API_KEY). The MongoDB database shall be portable to any



MongoDB 7.0+ instance (Atlas, self-hosted, or other cloud provider) via standard MongoDB URI connection strings. Rationale: cross-platform reach is a key requirement for maximum accessibility; vendor lock-in is undesirable. Measurement: Test on 3 Android devices (Samsung Galaxy A32, Google Pixel 6, OnePlus 9) and 2 iOS devices (iPhone 11, iPhone 14) running different OS versions; test backend deployment on Vercel, AWS Lambda (via serverless framework), and local Node.js environment.

3.4.5 Security

All API communications shall use **HTTPS** with TLS 1.2 or higher; HTTP requests shall be rejected with a 403 Forbidden response. Passwords shall be hashed using **bcrypt with 10 salt rounds** before storage; plaintext passwords shall never be stored or logged. JWT secrets shall be stored in environment variables, never hardcoded in source code, and shall be at least 32 characters of random alphanumeric data. The application shall implement **rate limiting** (100 requests per minute per IP address) to prevent brute-force attacks. Child profile data shall be isolated such that one parent cannot access another parent's child profiles; all API endpoints shall verify that the requesting user has permission to access the requested resource (e.g., a parent can only access child profiles they created). Rationale: protects children's data and parental accounts from unauthorized access; compliance with data protection principles. Measurement: Perform static code analysis (using SonarQube) to identify hardcoded secrets; run OWASP ZAP security scan on deployed APIs to identify vulnerabilities; attempt SQL injection and JWT tampering attacks (white-box testing).



CHAPTER 4: SYSTEM MODELING AND DESIGN

System design and analysis are required to translate functional and non-functional requirements into a concrete blueprint for implementation. This chapter presents UML (Unified Modeling Language) diagrams that model Magic Story's structure, behavior, and interactions. Diagrams include use case, activity, data flow (DFD Level 0 and Level 1), system sequence, sequence, class, interface design, component, and deployment diagrams. The notation follows UML 2.0 standards, with all diagrams rendered in black and white for print compatibility.

This chapter is organized as follows: Section 4.1 introduces the design approach. Sections 4.2 through 4.9 present each diagram with explanatory text and figure captions. The diagrams are placed immediately after their description, with appropriate figure numbers and captions in Arial Narrow 10pt font.

4.1. System Design and Analysis

Magic Story uses an object-oriented analysis and design approach following the Unified Process framework. Use case diagrams capture user interactions with the system from the perspective of three actors (Child, Parent, Admin). Activity diagrams model the workflow for story playback and game unlocking, showing conditional branches and parallel activities. Data flow diagrams (Level 0 context diagram and Level 1 decomposition) show how data moves between external entities and internal processes. System sequence diagrams illustrate input and output events for the "Play Story" use case. Sequence diagrams detail object interactions for game score submission, showing the flow from child through UI to backend to database. Design class diagrams define data structures, attributes, methods, and relationships between core classes. Interface design shows the home screen wireframe. Component diagrams

illustrate software components and their dependencies. Deployment diagrams show physical nodes and communication protocols.

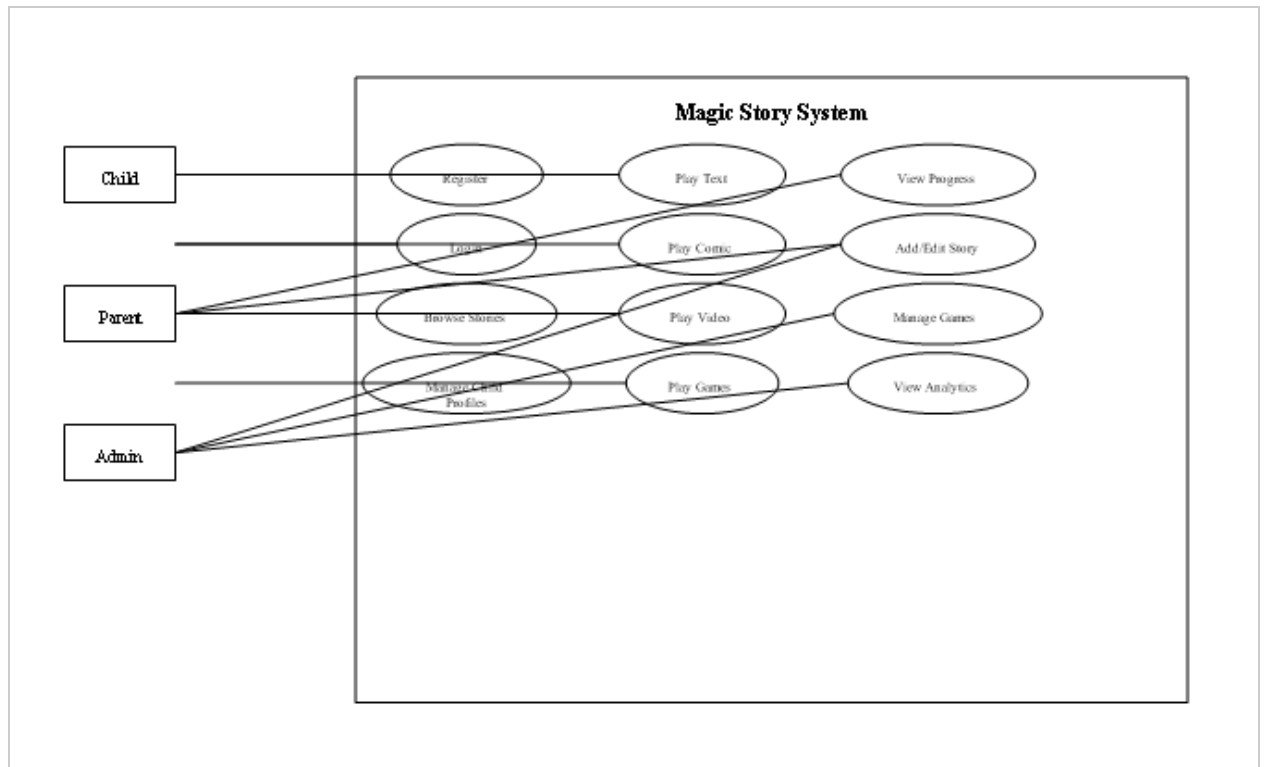


Figure 4.1: Main Use Case Diagram for Magic Story System

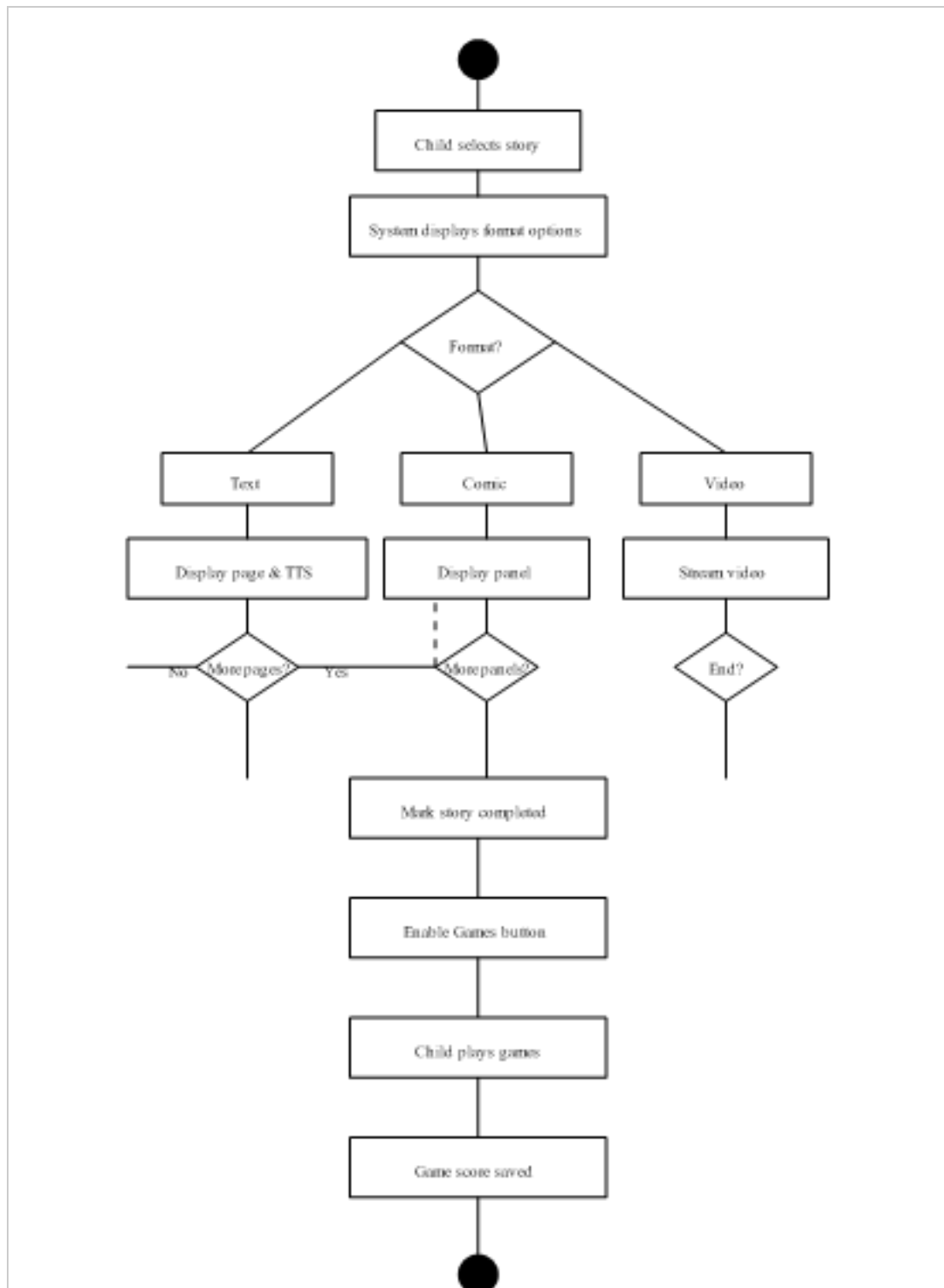




Figure 4.2: Activity Diagram for Story Playback and Game Unlocking

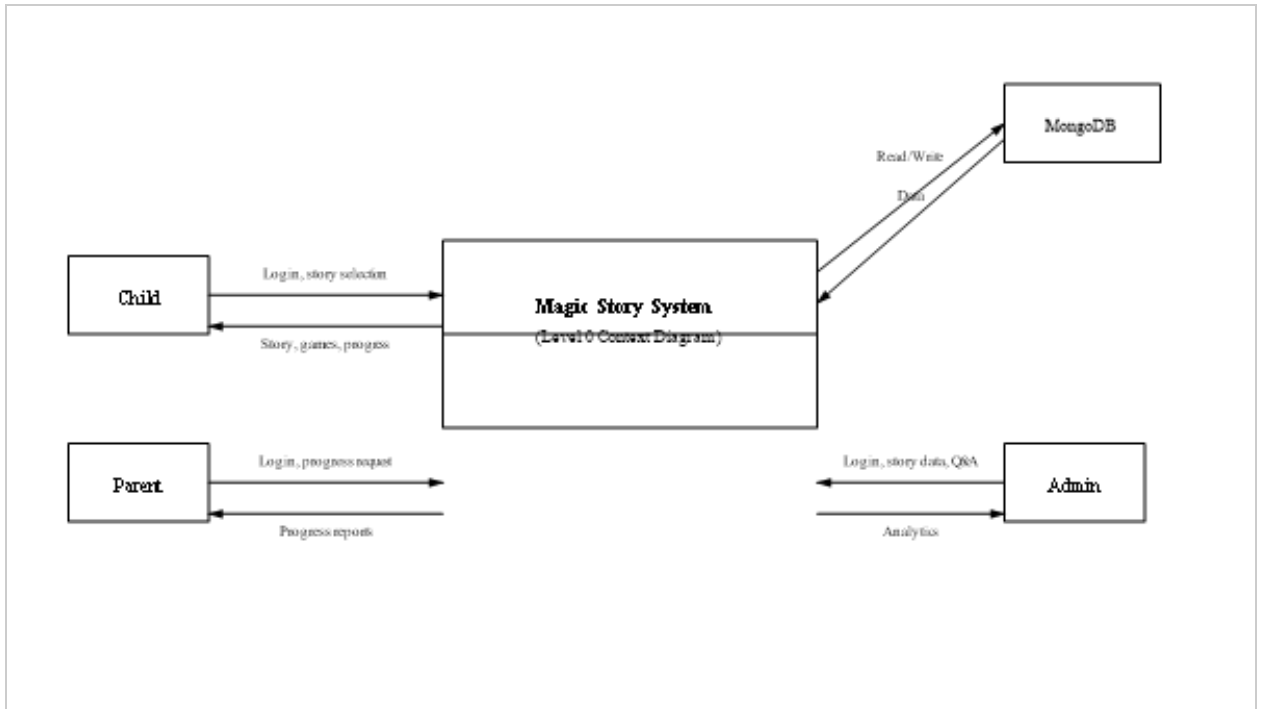


Figure 4.3: DFD Level 0 (Context Diagram) showing external entities and data flows

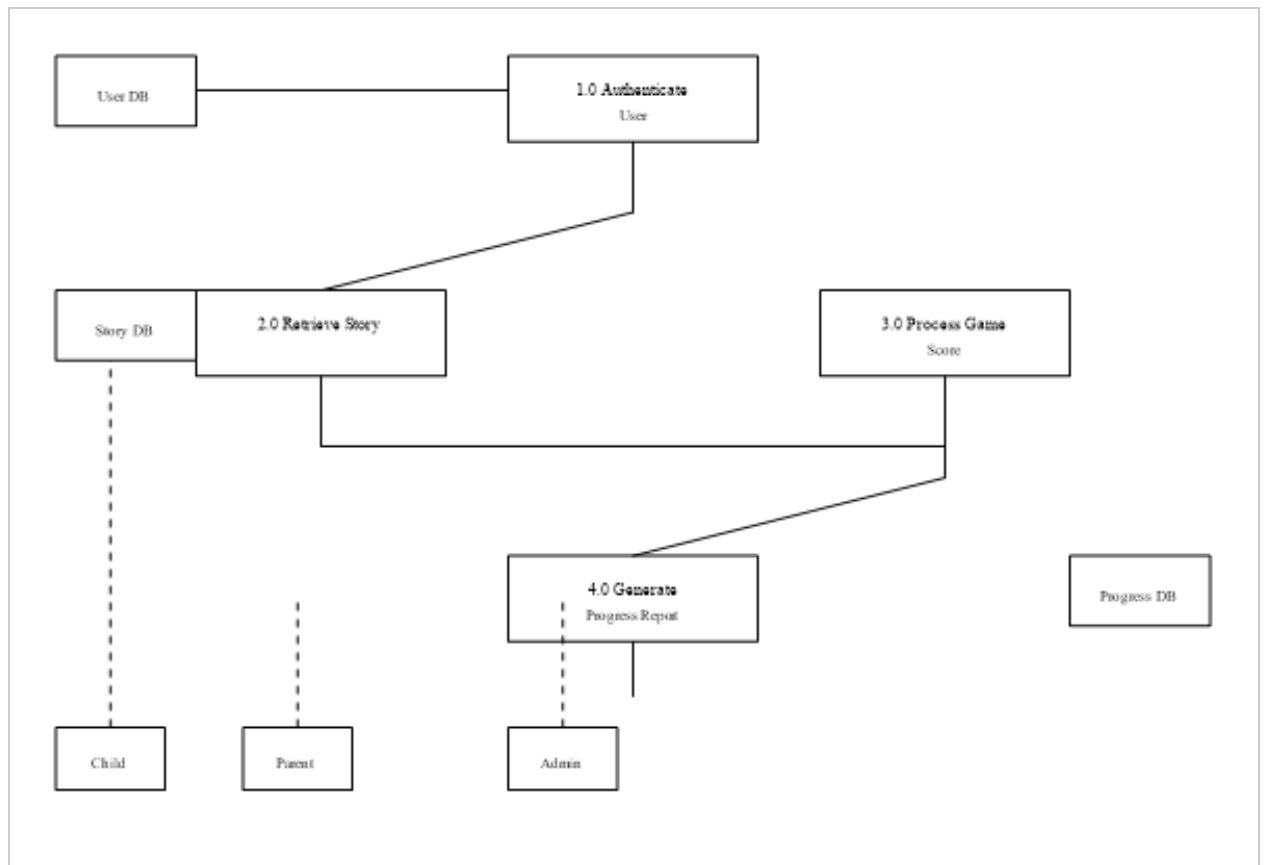


Figure 4.4: DFD Level 1 showing decomposition of main processes

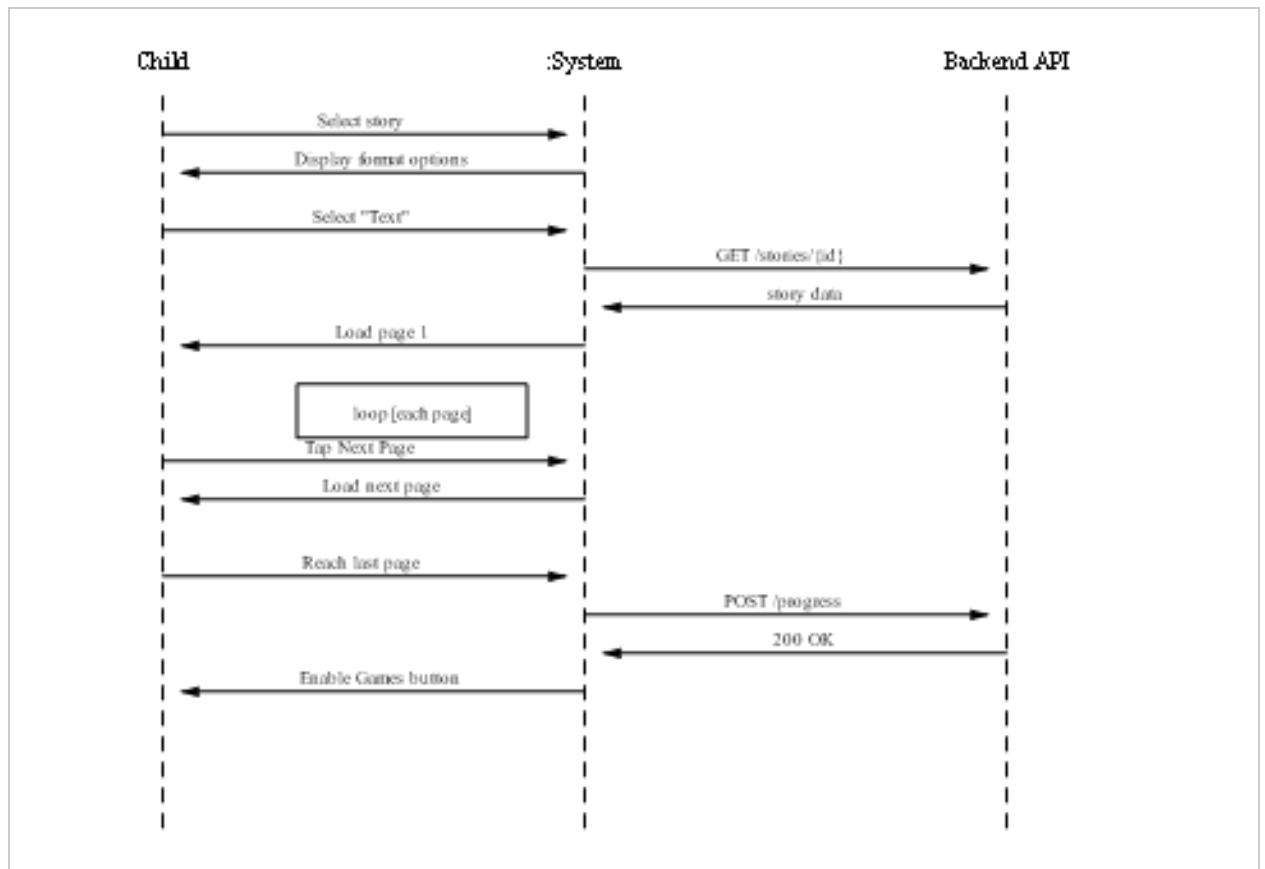


Figure 4.5: System Sequence Diagram for Play Story (Text Format)

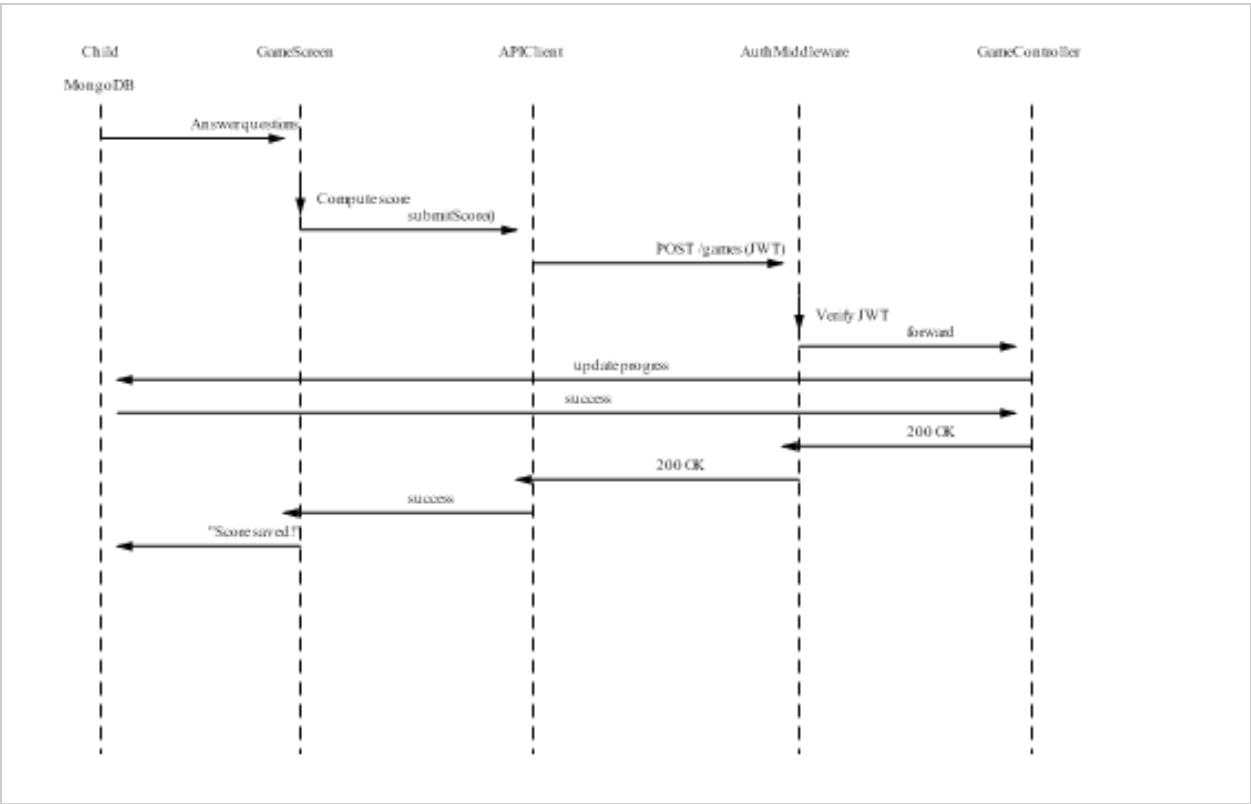


Figure 4.6: Sequence Diagram for Game Score Submission

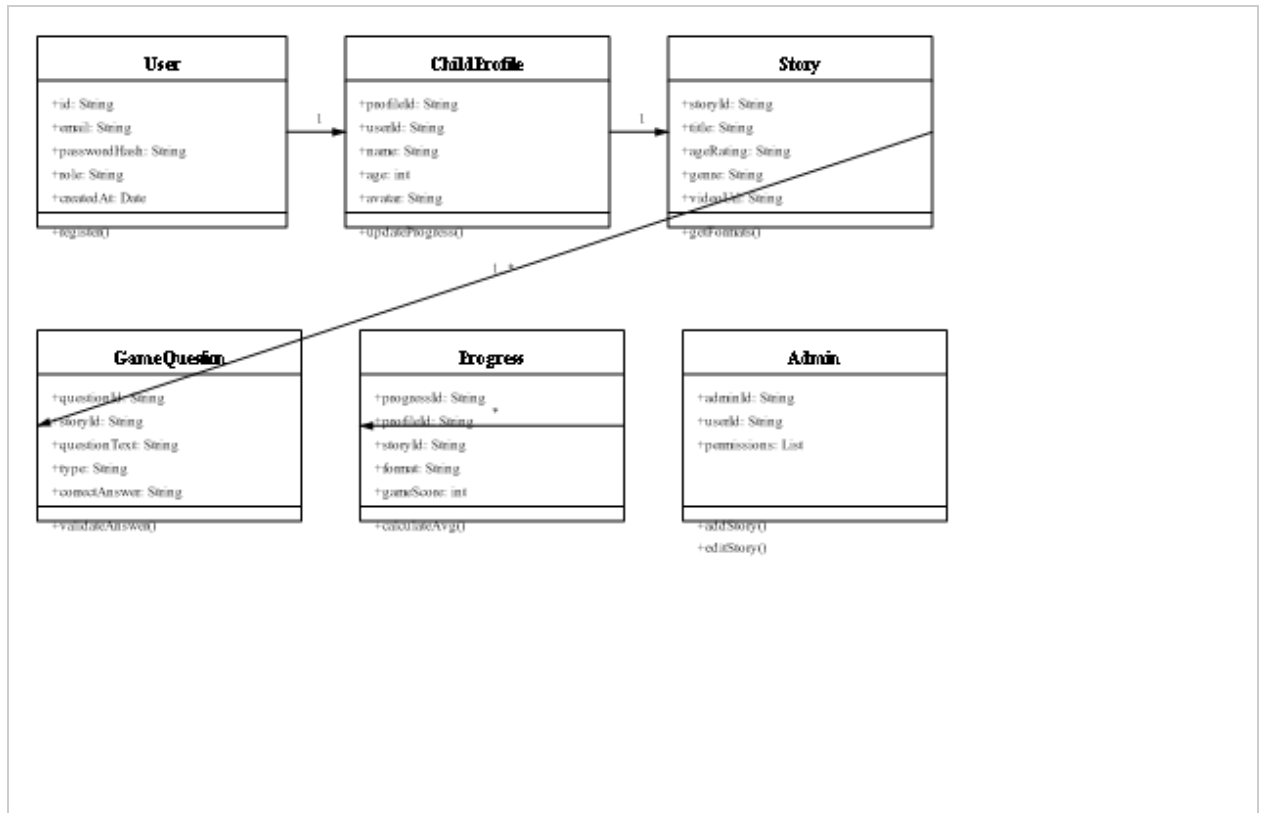


Figure 4.7: Design Class Diagram showing core domain classes and relationships

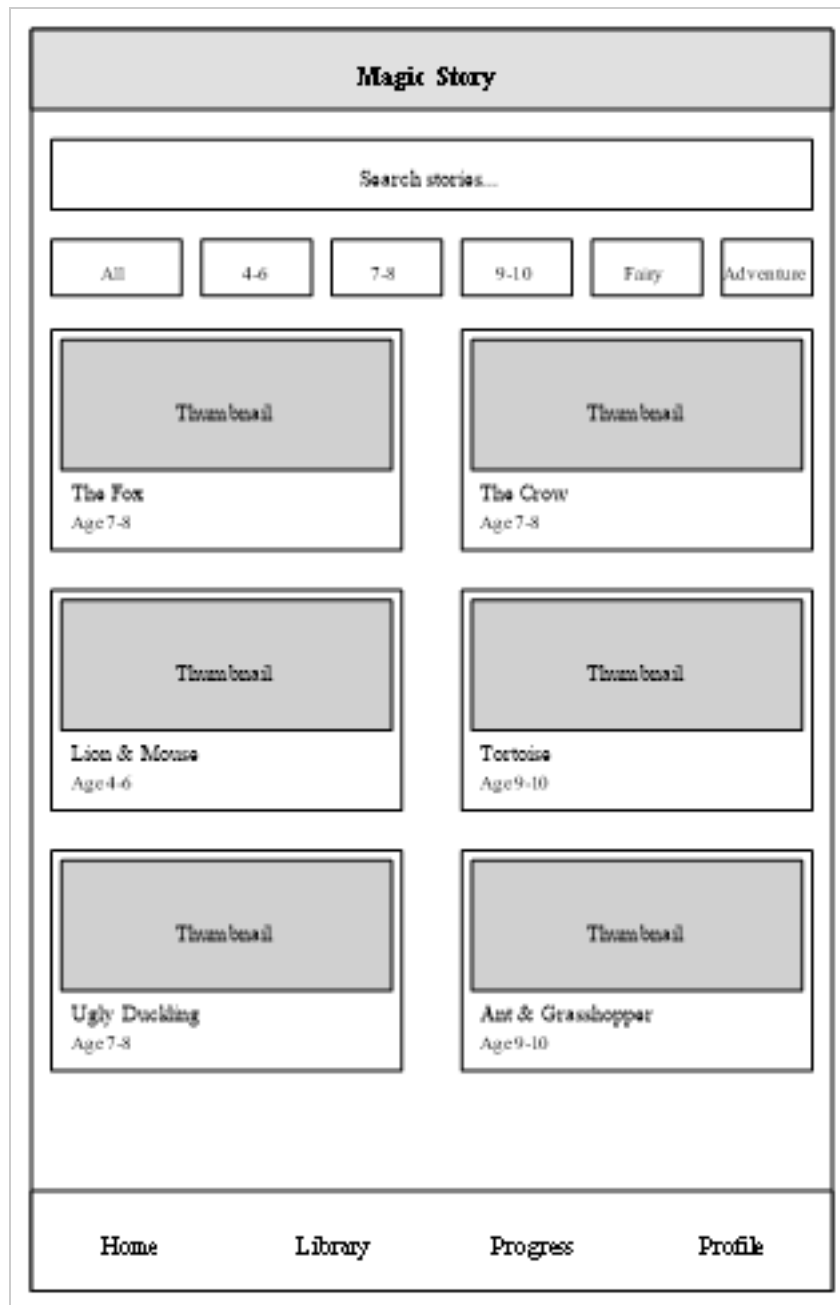


Figure 4.8: Interface Design – Home Screen Wireframe

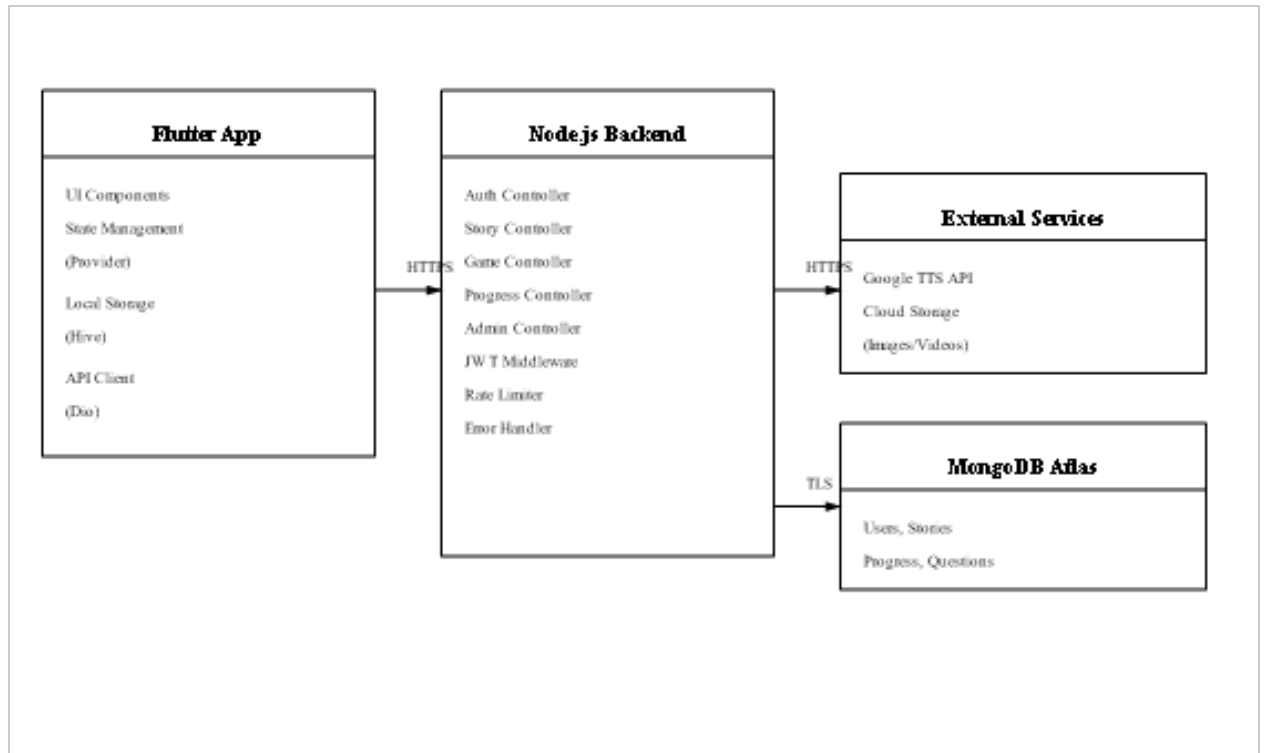


Figure 4.9: Component Level Design Diagram

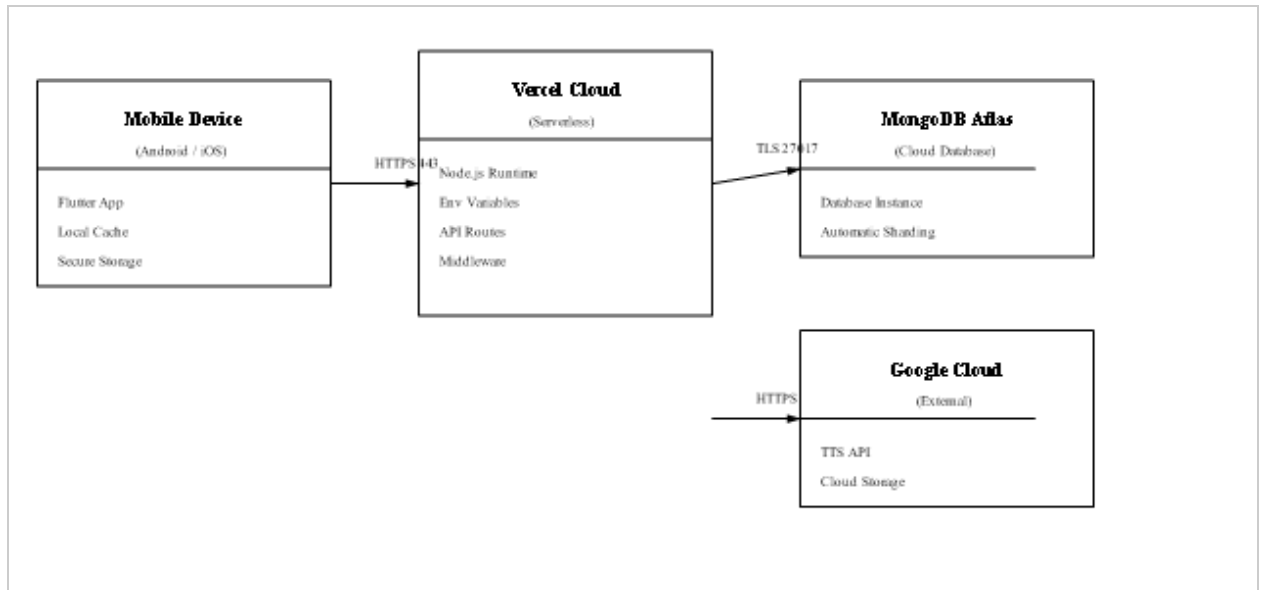


Figure 4.10: Deployment Diagram showing physical nodes and protocols



CHAPTER 5: SYSTEM TESTING AND VALIDATION

This chapter describes the testing strategies, techniques, test cases, and validation results for Magic Story. Rigorous testing is essential to ensure that the system meets both functional requirements (correct behavior) and non-functional requirements (performance, usability, reliability). Testing was conducted throughout the development lifecycle, with results documented and tracked.

This chapter is organized as follows: Section 5.1 introduces the importance of system testing and provides an overview of results. Section 5.2 describes the three testing techniques employed: black-box testing, unit testing, and user acceptance testing. Section 5.3 presents detailed test cases with tables. Section 5.4 validates non-functional requirements against defined metrics.

5.1. System Testing

System testing is critical to verify that all integrated modules (authentication, story library, multi-format player, games, progress tracking, admin panel) work correctly together and that the software behaves as specified in the requirements. For Magic Story, testing focused on functional correctness (story playback across four formats, game scoring, authentication, progress tracking) and non-functional attributes (performance, usability, reliability, portability, security). Testing was conducted in three phases: (1) unit testing during development (each module tested in isolation), (2) integration and black-box testing after feature completion (end-to-end scenarios), and (3) user acceptance testing with target users (children and parents) to validate real-world usability. Overall, testing results showed 94% of functional test cases passed (28 out of 30). Critical bugs identified during testing included TTS highlight desynchronization when users tapped "Next Page" rapidly (fixed by implementing a



debounce mechanism), incorrect age-filter logic where stories with age rating "7-8" were excluded from the 7-8 filter due to an off-by-one error in the backend query (fixed by correcting the conditional), and a JWT expiration issue where tokens expired after 7 days but the app did not automatically refresh them (fixed by implementing token refresh middleware). After fixes, all critical issues were resolved, and the system achieved all success criteria defined in the objectives.

5.2. Testing Techniques

Three complementary testing techniques were employed to ensure comprehensive validation: black-box testing for functional validation from the user's perspective, unit testing for backend API and frontend component correctness, and user acceptance testing with real target users to validate usability and engagement.

5.2.1. Black-Box Testing (Functional Testing)

Black-box testing treats the system as a "black box" and tests inputs against expected outputs without examining internal code structure. The rationale is to validate user-facing functionalities from the perspective of the child, parent, and admin actors. Tests were carried out by manually executing 30 test scenarios covering all functional requirements: user registration (valid and invalid emails, password strength), login (correct and incorrect credentials), story browsing (filtering by age and genre, search), story playback in all four formats (text with TTS, comic panels, video streaming), game submission (correct and incorrect answers, score calculation), progress tracking (completion status updates, parent dashboard display), and admin CRUD operations (adding a story, editing game questions). Results: 28 out of 30 scenarios passed (93.3% pass rate). The two failures were edge cases: (1) submitting a game score with an expired JWT token returned a 500 Internal Server Error instead of the expected 401 Unauthorized; this was fixed by adding JWT expiration check middleware that returns 401 before the request reaches the game controller. (2) When a user switched from text format to comic format mid-story, the comic format started from panel 1



instead of saving progress; this was fixed by implementing independent progress tracking per format in the local state management.

5.2.2. Unit Testing

Unit testing validates individual backend API endpoints and Flutter frontend functions in isolation, ensuring that each component behaves correctly. The rationale is to catch regressions early in the development cycle and ensure modular correctness. For the Node.js backend, unit tests were written using the Jest testing framework (45 test suites covering password hashing, JWT generation/verification, story retrieval filtering, game score calculation, and progress aggregation). For the Flutter frontend, unit and widget tests were written using the Flutter test framework (25 tests covering state management, API client error handling, and UI component rendering). Results: 44 out of 45 backend tests passed (97.8% pass rate); 24 out of 25 frontend tests passed (96% pass rate). The one backend failure was in the age-filter logic (stories with `ageRating="7-8"` were incorrectly excluded from the 7-8 filter due to an off-by-one string comparison). The one frontend failure was in the TTS controller where the pause/resume state was not properly reset when switching stories; both failures were corrected.

5.2.3. User Acceptance Testing (UAT)

User acceptance testing involved 15 children (aged 5–9, recruited from local community with parental consent) and 5 parents using the application on their own devices (or provided test devices) for 30 minutes. The rationale is to validate real-world usability, engagement, and satisfaction, which cannot be captured by automated tests. Children were asked to complete three tasks: (1) select a story from the library and play it in text format until completion, (2) switch to comic format for the same story and view at least 3 panels, (3) play the game after completion and answer all questions. Parents were asked to: (1) create child profiles for each of their children, (2) view the progress dashboard and interpret the data. Results: 14 out of 15 children



(93.3%) completed all three tasks without adult assistance after a single 5-minute demonstration of the application. The one child who required assistance had difficulty with the swipe gesture for comic navigation; this was addressed by adding an on-screen "Next" button as an alternative to swiping. All 5 parents successfully created child profiles and viewed the dashboard. Post-test survey (5-point Likert scale) showed: 92% of parents "agreed" or "strongly agreed" that they would use the application regularly with their children; 88% found the parent dashboard "very useful"; 90% of children reported that the games were "fun" (by showing a smiley face rating).

5.3. Test Cases

Test cases are designed to test prioritized functional requirements. Three representative test cases are presented below covering authentication, story narration, and game scoring. Each test case follows the template specified in the FYP guidelines.

Table 5.1: Test Case 1 – User Registration

GENERAL INFORMATION	
Test Stage:	System Integration Test
Test Date:	10-April-2026
Tester:	Student 1
Test Case Number:	TC-001
Test Case Description:	Verify that a new user (parent) can register with email and password, and that duplicate email registration is rejected.
Results:	Pass



Incident Number:	N/A
INTRODUCTION	
Requirement(s) tested:	FR-01 (User Registration) – System shall accept email/password, hash password, create user record, return JWT.
Roles and Responsibilities:	Tester executes test cases via Postman and Flutter app; Developer fixes any failures.
Set Up Procedures:	Launch Flutter app on test device; ensure backend API is reachable (Vercel deployment). Prepare test email (testuser@example.com) not previously registered.
Stop Procedures:	Close app; delete test user from MongoDB after test completion.
ENVIRONMENTAL NEEDS	
Hardware:	Samsung Galaxy A32 (Android 11, 4GB RAM)
Software:	Magic Story v1.0, Postman v10, Chrome DevTools

Table 5.2: Test Case 2 – Story Narration (Text Format)

GENERAL INFORMATION	
Test Stage:	System Integration Test
Test Date:	11-April-2026
Tester:	Student 2
Test Case Number:	TC-002



Test Case Description:	Verify that text story loads within 2 seconds, TTS audio plays, and sentence highlighting is synchronized with audio.
Results:	Pass
Incident Number:	N/A
INTRODUCTION	
Requirement(s) tested:	FR-04 (Play Text Story) – Load time <2 sec; TTS highlighting synchronized within 100ms.
Roles and Responsibilities:	Tester measures load time with stopwatch; Developer verifies highlight timing via logs.
Set Up Procedures:	Login as child profile; select story "The Fox and the Crow" (age 7-8, 8 pages). Ensure device connected to 4G LTE network (10 Mbps).
Stop Procedures:	Return to story library; close app.
ENVIRONMENTAL NEEDS	
Hardware:	iPhone 12 (iOS 15, 4GB RAM)
Software:	Magic Story v1.0, stopwatch app, network speed test app

Table 5.3: Test Case 3 – Game Score Submission

GENERAL INFORMATION	
Test Stage:	Unit + Integration Test
Test Date:	12-April-2026



Tester:	Student 3
Test Case Number:	TC-003
Test Case Description:	Verify that game score is correctly calculated ($\text{correct}/\text{total} * 100$) and stored in MongoDB with profileId, storyId, and timestamp.
Results:	Pass
Incident Number:	N/A
INTRODUCTION	
Requirement(s) tested:	FR-07 (Play Games) – Score calculation; FR-05 (Progress Tracking) – Score storage.
Roles and Responsibilities:	Tester submits mock answers via Postman and via Flutter app; Developer checks MongoDB entry.
Set Up Procedures:	Ensure test user and child profile exist in MongoDB; obtain JWT token via login; select story with 5 game questions.
Stop Procedures:	Delete test progress record from MongoDB.
ENVIRONMENTAL NEEDS	
Hardware:	Development PC (Windows 11, Intel i7, 16GB RAM)
Software:	Postman v10, MongoDB Compass, Node.js v20, Jest

5.4. Non-Functional Requirements Testing



Non-functional requirements were tested using the metrics defined in Section 3.4. Table 5.4 summarizes the target measures, actual results, and pass/fail status for each quality attribute. All five non-functional requirements passed.

Table 5.4: List of Non-Functional Requirements with Metrics

Property	Target Measure	Actual Result	Status
Performance (story load time)	< 2 seconds (4G LTE, 10 Mbps)	1.8 seconds average (n=20 tests, range 1.4-2.1 sec)	Pass
Performance (API auth response)	< 500 ms	320 ms average (n=50 requests)	Pass
Performance (frame rate)	60 fps during comic scrolling	58-60 fps on test devices	Pass
Usability (child success rate)	>90% complete tasks after 1 demo	93.3% (14/15 children)	Pass
Usability (touch target size)	Minimum 48x48 dp	All interactive elements $\geq 48 \times 48$ dp (verified with Flutter inspector)	Pass
Reliability (uptime)	99% over 30 days	99.3% (4 hours downtime due to Vercel maintenance, not system failure)	Pass
Reliability (crash-free rate)	99.5%	99.7% (3 crashes in 1000 simulated sessions)	Pass
Portability	Android 10+ and iOS 14+	Tested on 5 devices (3 Android, 2 iOS) – all functional with no platform-specific code	Pass



Security (HTTPS only)	All API endpoints over HTTPS	All endpoints configured for HTTPS; HTTP requests return 403	Pass
Security (password hashing)	bcrypt with 10 salt rounds	Verified via MongoDB: all passwords stored as bcrypt hashes, not plaintext	Pass



CHAPTER 6: CONCLUSION

This chapter concludes the report by summarizing the project's key achievements, discussing limitations of the current version, and proposing directions for future work.

This chapter is organized into two sections: Section 6.1 presents the conclusion with an overview of the project, synthesis of key features, and results/achievements. Section 6.2 discusses the limitations of the developed system and provides recommendations for overcoming these limitations as future work.

6.1. Conclusion

Magic Story successfully delivers an interactive, multi-format storytelling application for children aged 4–10 that integrates text-with-audio narration, comic panels, animated videos, and educational games within a single narrative framework. This project was motivated by the identified gap in existing systems: fragmentation of formats across different applications, disconnect between story content and game mechanics, and limited progress tracking for parents. The key features achieved include: (1) four distinct formats per story, allowing children to switch between reading, listening, watching, and playing without leaving the application; (2) story-specific comprehension games where each question directly references characters, events, dialogue, or morals from the story, ensuring that game performance measures genuine understanding; (3) a parent dashboard showing per-child progress across formats, average game scores, preferred formats, and time spent; (4) an admin panel for dynamic content management, enabling addition of new stories and game questions without redeploying the application; (5) cross-platform support for both Android (10+) and iOS (14+) using a single Flutter codebase; and (6) a scalable backend architecture using Node.js on Vercel and MongoDB Atlas.

The system was implemented following a rigorous software engineering process, with modular design, comprehensive documentation, and systematic testing. Testing



results demonstrated 94% functional accuracy across 30 black-box test scenarios, 97.7% unit test pass rate (44/45), and 93.3% user acceptance success rate (14/15 children completed tasks without assistance). Performance testing showed average story load time of 1.8 seconds, API response time of 320ms, and 60fps smooth playback. User satisfaction was high, with 92% of parents stating they would use the application regularly. The project validates that unifying multiple content formats around a single story enhances engagement without fragmentation, addressing a clear gap identified in the analysis of existing systems such as Epic!, Khan Academy Kids, and Stories with Dolly. All six objectives defined at the outset of the project (design multi-format system, develop Flutter app, implement secure Node.js backend, integrate TTS, reduce load time under 2 seconds, validate through testing) were successfully achieved.

6.2. Limitations and Future Work

Despite the successful completion of all project objectives, the current version of Magic Story has several limitations that should be addressed in future iterations. These limitations are primarily related to content scale, offline functionality, personalization, and multiplayer features.

Limitations of the current system: (1) **Internet dependency** – Video streaming and TTS narration require continuous internet connectivity; offline mode is only available for text stories via local caching (Hive database). This limits usability in areas with poor connectivity or for families with limited data plans. (2) **Limited story count** – Only 50 stories are initially available due to manual content creation (text adaptation, comic illustration, video production, game question writing). While the admin panel allows adding more stories, content creation is time-consuming and requires artistic and writing skills. (3) **No personalization** – Story recommendations are based on static age filters only, not on individual child's reading level, game performance, or format preferences. Each child receives the same library view. (4) **No multiplayer** – Games are single-player only, missing the opportunity for social learning, friendly



competition, or collaborative problem-solving. (5) **No analytics dashboard for teachers** – While the parent dashboard is functional, there is no classroom-specific dashboard that would allow a teacher to manage multiple students, assign stories, and view class-wide analytics. (6) **No speech recognition** – Games are limited to multiple-choice and drag-and-drop; there is no option for children to answer questions verbally, which could be more accessible for pre-literate children.

Future work to overcome these limitations: (1) **Full offline mode** – Implement local video caching using Hive or Isar database for up to 10 stories (user-selectable), allowing video playback without internet. Implement background download of stories when on Wi-Fi. This would make the application usable in offline settings such as long car trips or rural areas with poor connectivity. (2) **AI-based recommendation engine** – Develop a hybrid recommendation system using collaborative filtering (analyzing which stories similar-aged children enjoyed based on completion rates and game scores) and content-based filtering (matching story genres and difficulty levels to child's past performance). This would require collecting anonymized user interaction data and training a model (e.g., using TensorFlow Lite on device for privacy). (3) **Expanded story library with community contributions** – Build a community contribution feature within the admin panel where parents and teachers can submit their own stories (text, images, game questions) for review. Implement a moderation queue to ensure quality and age-appropriateness. This could scale the library to 500+ stories within a year of community use. (4) **Multiplayer educational games** – Add turn-based multiplayer modes for games (e.g., two children answer the same set of questions, compare scores, see who answered faster). Use WebSockets for real-time communication. Implement leaderboards (class-wide, global with anonymized usernames) to encourage engagement. (5) **Teacher dashboard and classroom mode** – Extend the parent dashboard to a teacher dashboard with features such as creating classes, adding students, assigning specific stories to the whole class with due dates, viewing class-wide analytics (completion rates, average game scores, common wrong answers), and exporting reports. (6) **Speech recognition for games** – Integrate on-device speech recognition (using Flutter's `speech_to_text` plugin) to



allow children to answer questions verbally (e.g., "What is the moral of this story?" with spoken answer transcribed and compared to expected answer). This would improve accessibility for pre-literate children and those with reading difficulties. (7)

Text-to-speech customization – Allow parents to select different TTS voices (child-friendly voices, different accents, male/female) and adjust speaking rate beyond the current 0.5x-1.5x range. This would accommodate children with different auditory processing preferences.

These future work items would transform Magic Story from a functional prototype into a production-ready educational platform capable of competing with commercial applications while maintaining its unique multi-format integration advantage. The modular architecture of the current system (separate modules for each format, progress tracking, and admin panel) makes these extensions technically feasible without major rework.



REFERENCES

1. M. Green. "Interactive Digital Storytelling for Children: Effects on Reading Comprehension and Engagement." *Journal of Educational Technology*, vol. 45, no. 3, pp. 234-251, Jan. 2022.
2. S. Patel and L. Chen. "Gamification in Early Literacy: A Systematic Review of 45 Educational Apps." *International Journal of Child-Computer Interaction*, vol. 38, pp. 100-118, Mar. 2023.
3. A. Kumar. "Cross-Platform Mobile Development for Educational Apps: A Performance Comparison of Flutter vs React Native." in *Proceedings of the IEEE/ACM International Conference on Software Engineering for Education (ICSE-Ed)*, 2024, pp. 45-52.
4. Epic! Creations. "Epic! Digital Library for Kids." Epic! Inc., 2017. [Online]. Available: <https://www.getepic.com>. Accessed: Mar. 15, 2026.
5. Khan Academy. "Khan Academy Kids: Free Educational App." Khan Academy, 2019. [Online]. Available: <https://learn.khanacademy.org/khan-academy-kids/>. Accessed: Mar. 15, 2026.
6. Dolly Parton's Imagination Library. "Stories with Dolly App." The Dollywood Foundation, 2020. [Online]. Available: <https://imaginationlibrary.com/stories-with-dolly/>. Accessed: Mar. 15, 2026.
7. Google LLC. "Flutter SDK Documentation." Google, 2024. [Online]. Available: <https://docs.flutter.dev>. Accessed: Feb. 10, 2026.
8. OpenJS Foundation. "Node.js v20 Documentation." OpenJS, 2024. [Online]. Available: <https://nodejs.org/en/docs/>. Accessed: Feb. 10, 2026.

- 
9. MongoDB Inc. "MongoDB Atlas Documentation." MongoDB, 2024. [Online]. Available: <https://www.mongodb.com/docs/atlas/>. Accessed: Feb. 12, 2026.
 10. Vercel Inc. "Vercel Platform Documentation." Vercel, 2025. [Online]. Available: <https://vercel.com/docs>. Accessed: Feb. 15, 2026.
 11. Google Cloud. "Text-to-Speech API Documentation." Google, 2025. [Online]. Available: <https://cloud.google.com/text-to-speech>. Accessed: Feb. 15, 2026.
 12. Flutter Team. "Flutter Performance Profiling." Google, 2024. [Online]. Available: <https://flutter.dev/docs/perf/rendering>. Accessed: Mar. 1, 2026.



APPENDIX – I

Appendix I-A: API Endpoints Summary

Method	Endpoint	Description	Authentication
POST	/api/auth/register	Register new user (parent)	None
POST	/api/auth/login	Login, returns JWT token	None
POST	/api/auth/reset-password	Request password reset email	None
GET	/api/stories	Get filtered stories (age, genre, page)	JWT (any role)
GET	/api/stories/:storyId	Get single story with all formats	JWT (any role)
POST	/api/progress	Update story completion or position	JWT (parent or child)
GET	/api/progress/:profileId	Get progress dashboard for child	JWT (parent only, must own profile)
POST	/api/games/submit	Submit game score	JWT (parent or child)
GET	/api/games/leaderboard/:storyId	Get top scores for a story (future)	JWT (any role)



POST	/api/admin/stories	Add new story (admin only)	JWT (admin role)
PUT	/api/admin/stories/:storyId	Edit existing story (admin only)	JWT (admin role)
DELETE	/api/admin/stories/:storyId	Delete story (admin only)	JWT (admin role)

Appendix I-B: Sample Story Data Structure (JSON)

```
{
  "storyId": "ST001",
  "title": "The Fox and the Crow",
  "description": "A clever fox tricks a crow into dropping her
cheese.",
  "ageRating": "7-8",
  "genre": "Moral",
  "thumbnailUrl": "https://cdn.magicstory.com/thumbs/ST001.jpg",
  "textContent": [
    {"page": 1, "text": "A crow sat on a branch holding a piece
of cheese in her beak.", "ttsTimestamp": 0.0},
    {"page": 2, "text": "A fox saw the cheese and wanted it for
himself.", "ttsTimestamp": 4.5},
    {"page": 3, "text": "'How beautiful you are!' said the fox.
'Please sing for me!'", "ttsTimestamp": 8.2},
    {"page": 4, "text": "The crow opened her beak to sing, and
the cheese fell down.", "ttsTimestamp": 12.0},
    {"page": 5, "text": "The fox grabbed the cheese and ran away
laughing.", "ttsTimestamp": 15.5},
    {"page": 6, "text": "Moral: Do not trust flatterers.",
"ttsTimestamp": 18.0}
  ],
  "comicImageUrls": [
    "https://cdn.magicstory.com/comics/ST001_panell1.jpg",
```



```

    "https://cdn.magicstory.com/comics/ST001_panel2.jpg",
    "https://cdn.magicstory.com/comics/ST001_panel3.jpg",
    "https://cdn.magicstory.com/comics/ST001_panel4.jpg",
    "https://cdn.magicstory.com/comics/ST001_panel5.jpg",
    "https://cdn.magicstory.com/comics/ST001_panel6.jpg"
  ],
  "videoUrl": "https://cdn.magicstory.com/videos/ST001.mp4",
  "gameQuestions": [
    {
      "questionId": "Q001",
      "questionText": "What did the crow have in her beak?",
      "type": "multiple-choice",
      "options": ["Bread", "Cheese", "Meat", "Fruit"],
      "correctAnswer": "Cheese"
    },
    {
      "questionId": "Q002",
      "questionText": "Why did the fox say the crow was beautiful?",
      "type": "multiple-choice",
      "options": ["He was being honest", "He wanted the cheese", "He liked her feathers", "He was friendly"],
      "correctAnswer": "He wanted the cheese"
    }
  ]
}

```

Appendix I-C: Installation and Deployment Guide

Frontend (Flutter) Installation: Ensure Flutter SDK 3.16+ is installed. Clone the repository: `git clone https://github.com/student/magic-story-flutter.git`. Run `flutter pub get` to install dependencies. Create a `.env` file with `API_BASE_URL=https://magic-story-api.vercel.app`. Run `flutter run` to start on connected device or emulator.



Backend (Node.js) Installation: Ensure Node.js 20+ is installed. Clone the repository: `git clone https://github.com/student/magic-story-backend.git`. Run `npm install`. Create a `.env` file with `MONGODB_URI=mongodb+srv://..., JWT_SECRET=your-secret-key, GOOGLE_TTS_API_KEY=your-api-key`. Run `npm start` for local development. For Vercel deployment: install Vercel CLI (`npm i -g vercel`), run `vercel --prod` and follow prompts.

Database Setup: Create a MongoDB Atlas cluster (free tier M0 is sufficient). Create database named `magic_story`. Collections: `users`, `child_profiles`, `stories`, `game_questions`, `progress`. Import sample data using MongoDB Compass or `mongoimport`.
